

数学建模 · 国赛速成课

——2 小时掌握通用解题套路

数学建模老哥独家出品

2026 年 6 月 18 日

数学建模老哥

目录

1 入门：全国大学生数学建模竞赛 (CUMCM)	6
1.1 比赛介绍	6
1.2 比赛内容与题型	6
1.3 出题规律	7
1.4 获奖率与评奖机制	8
1.5 比赛流程（三天 74 小时）	9
1.6 必备工具	11
1.7 新手必知的三条生存法则	12
1.8 赛前速查清单	13
2 评价类模型	13
2.1 什么时候用	13
2.2 四选一速查	13
2.3 五步解题流程	14
2.4 案例：供应商综合评价	15
2.5 代码模板	19
2.6 常见坑	20
3 分类模型	21
3.1 什么时候用	21
3.2 四选一速查	21
3.3 五步解题流程	21
3.4 案例：信用卡欺诈检测与推导	22
3.5 代码模板	25
3.6 常见坑	26
4 聚类模型	26

4.1	什么时候用	26
4.2	三选一速查	27
4.3	四步解题流程	27
4.4	案例：电商客户分群	28
4.5	代码模板	30
4.6	常见坑	31
5	预测模型	31
5.1	什么时候用	31
5.2	四选一速查	32
5.3	五步解题流程	32
5.4	案例：产品销量预测与推导	32
5.5	代码模板	36
5.6	常见坑	37
6	优化模型	38
6.1	什么时候用	38
6.2	四选一速查	38
6.3	四步解题流程	38
6.4	案例：生产计划优化与推导	39
6.5	代码模板	42
6.6	常见坑	43
7	机理模型	43
7.1	什么时候用	43
7.2	三选一速查	44
7.3	四步解题流程	44
7.4	案例：传染病传播预测（SIR 模型）	44
7.5	代码模板	48
7.6	常见坑	49

8 LaTeX 写作——使用 CUMCM 模板	49
8.1 为什么用 LaTeX	49
8.2 使用流程（5 步上手）	49
8.3 模板文件结构与各文件填写指南	50
8.4 排版高频代码（复制即用）	55
8.5 十大编译错误与快速排错	56
8.6 引用参考文献	57
8.7 赛前必验证项	57
9 Word 写作速成	57
9.1 为什么 Word 也是正经选择	57
9.2 第一步：设置页面和样式（比赛开始前就做好）	58
9.3 第二步：三线表（国赛表格唯一规范）	59
9.4 第三步：公式排版	60
9.5 第四步：图片插入	61
9.6 第五步：页码和页眉	62
9.7 第六步：目录	62
9.8 第七步：参考文献	62
9.9 第八步：最终格式检查清单	63
9.10 Word 最易翻车的 5 个问题	64
10 摘要写作要点	64
10.1 摘要的重要性与评审视角	65
10.2 标准五段式结构与写作模板	65
10.3 各步骤的常见错误与规范写法	66
10.4 完整摘要范例（A 题——物理/工程类）	67
10.5 摘要用词的规范性要求	68
10.6 关键词选择规范	68
10.7 常见扣分项与急救措施	69
10.8 摘要提交流程与自检清单	69

11 团队分工与时间管理	70
11.1 三天标准时间线	70
11.2 三人角色分工	71
11.3 最致命的三个配合问题	72
11.4 协作工具	72
11.5 紧急预案	73
12 AI 结合数模	73
12.1 AI 在国赛中的定位	73
12.2 赛题分析：怎么用 AI 分析题目	74
12.3 代码生成：怎么用 AI 写代码不出错	75
12.4 绘图：怎么用 AI 画图	76
12.5 论文润色：怎么用 AI 改论文	77
12.6 不可逾越的红线	77
12.7 推荐 AI 工具组合	78
13 国赛易违规条款分析	78
13.1 查重红线（碰了就淘汰）	78
13.2 引用规范	79
13.3 代码查重（最容易忽略的雷区）	79
13.4 格式违规（不起眼但扣分）	80
13.5 提交环节（最后一公里翻车）	81
13.6 违规后果速查	81

一句话：国赛 = 三人组队 + 三天（74 小时）+ 一篇论文。比的不是谁算得对，是谁的论文让评委觉得”有道理、有水平、做得完”。

1.1 比赛介绍

全国大学生数学建模竞赛（CUMCM，简称”国赛”）由中国工业与应用数学学会主办，始于 1992 年，每年 9 月举行，是世界上规模最大的数学建模竞赛。2024 年参赛队伍超过 **6 万支**（约 18 万名学生），覆盖全国所有 985/211 高校及绝大部分本科院校。

比赛性质：

- 本科生组（本科组）和专科生组（专科组）分开评奖
- 三人一组，不限专业（通常组合：数学/统计 + 计算机 + 工程/经管）
- 赛题发布后 74 小时内提交论文（PDF + 附件）
- 论文为唯一评审依据——不需要现场答辩（国一除外）

参赛价值：

- 保研加分：绝大多数高校将国赛纳入保研加分体系，国奖加分权重最高
- 能力证明：建模能力 + 编程能力 + 写作能力的三合一训练
- 竞赛经历：在求职/申请中，国赛经历被广泛认可

1.2 比赛内容与题型

国赛每年出 4-5 道题（A/B/C/D/E 题），参赛队任选一题。题目来源于实际工程、经济、管理、环境等领域，题目类型分为以下几类：

表 1: 国赛题型与典型题目

题号	倾向	典型场景	涉及方法
A 题	物理/工程	热传导、应力分析、天线设计、CT 成像	微分方程、有限元、数值模拟

表 1: 国赛题型与典型题目 (续)

题号	倾向	典型场景	涉及方法
B 题	社会/经济	交通优化、资源分配、政策评估、调度	优化模型、图论、博弈论
C 题	数据/统计	分类识别、预测、综合评价、异常检测	机器学习、统计建模、数据分析
D 题	优化/运筹	路径规划、生产调度、库存管理	线性/整数规划、启发式算法
E 题	综合/交叉	环境评估、生态建模、传染病、能源	机理 + 数据混合建模

选题策略：

- C 题（数据题）选的人最多（约占 40%），门槛低但区分度小——很难脱颖而出拿国一
- A/B 题选的人相对少（各约占 20%-25%），区分度大——做得好容易冲国奖，但难度高
- 建议：队伍中有物理/工科背景 → 优先 A/B 题；纯数学/统计/计算机 → C 题最稳；跨学科组合 → B 或 E 题

1.3 出题规律

分析近 10 年（2015-2024）国赛真题，可以得到以下规律：

规律一：三年不重复，但有周期

同一领域的题目不会连续出现。例如 2019 年 B 题是物理类（同心鼓）、2020 年 B 题是社会类（沙漠游戏）、2021 年 B 题是优化类（乙醇偶合制备 C4 烯烃）。但每 3-4 年，类似题型会以新面貌回归。

规律二：热点映射

当年的社会/科技热点大概率进入赛题：

- 2020 年（新冠元年）：多道题涉及传染、隔离、资源调配等场景
- 2022 年（碳中和热点）：涉及能源优化、碳减排评估
- 2024 年：AI/大数据相关的数据建模题目比重增加

规律三：数据题（C 题）年年有

C 题自 2015 年固定为大数据/统计建模方向，每年必有一道纯数据题。题目通常给出一个较大的 Excel/CSV 附件，要求进行数据处理、建模、预测或分类。

规律四：每道题通常包含 3-4 个小问

小问设计遵循递进逻辑：第 1 问较简单（入门/验证），第 2-3 问逐渐复杂，第 4 问（如有）通常要求模型改进或灵敏度分析。前两问做不完不影响后两问——评分是分问计分。

针对性的准备建议：

- 赛前必看最近 3 年的 A/B/C 题真题和优秀论文（了解出题风格和评审标准）
- 提前积累各领域的常见模型代码模板（本讲义第 2-7 章覆盖了 6 大类）
- 准备一套通用的论文模板（本讲义第 8-9 章）

1.4 获奖率与评奖机制

国赛实行**省级评选** + **全国送审**的两级评审机制。

表 2: 国赛获奖比例与评审流程

奖项等级	大致比例	评审方	说明
省一等奖	约 8%	省组委会	省一中的前 10%-15% 送审全国
省二等奖	约 15%	省组委会	直接确定，不送国评
省三等奖	约 20%	省组委会	直接确定
成功参赛奖	剩余约 57%	—	提交有效论文即可获得
全国一等奖	约 0.5%-1%	全国组委会	需答辩（部分年份线上）
全国二等奖	约 2%-3%	全国组委会	论文评审，不需答辩

关键数字解读：

- 提交了论文 → **基本都有**“成功参赛奖”（等同于完成比赛）
- 约 43% 的队伍能拿到省三或以上
- 约 8% 拿省一，其中约 1% 最终拿国一，约 2% 拿国二
- 也就是说：**每 100 支队中约 3 支队拿到国奖**

评审流程：

1. 省组委会初筛：排除格式违规、查重超标（全文重复率 > 25% 或代码重复率 > 50%）
2. 评委分组评审：每篇论文由 2-3 位评委独立打分
3. 省一产生后：前 10%-15% 的省一论文送全国组委会复审

4. 全国复审：重新打分，产生国一和国二
5. 国一候选：需参加答辩（线下或线上，回答关于论文的提问）

评分的核心维度（按权重排序）：

1. 模型的合理性（30%-40%）：模型是否符合问题特征？假设是否合理？
2. 求解的正确性（25%-30%）：结果是否正确？是否完成了所有小问？
3. 论文的表达（20%-25%）：结构是否清晰？逻辑是否通顺？图表是否规范？
4. 创新性（10%-15%）：是否有独到的建模思路？是否有模型改进？

评委最看重的不是“用了多高深的算法”，而是——为什么选这个模型？数据怎么处理的？假设合不合理？灵敏度分析做了没有？结论有没有数据支撑？

1.5 比赛流程（三天 74 小时）

例子与推导

第一天（周四 18:00 — 周五 01:00）

18:00-18:30：拿到题目，快速通读

- 下载所有题目和附件，三人各自快速通读所有题目（不求深，先求全）
- 每人标注自己认为“能做”“做不了”“需要查资料”的题目

18:30-20:00：讨论选题

- 三人各自花 20 分钟深入阅读 1-2 道候选题目，列出“会用到什么方法”“需要什么数据”“有什么难点”
- 碰头讨论，汇总信息，确定候选题目（通常 2 选 1 或 3 选 1）

20:00-22:00：定题 + 查文献

- 最晚 22:00 必须定题。超过 22:00 还没定题说明团队讨论效率有问题
- 定题后立刻开始查文献（知网/Google Scholar），搜类似问题的建模方案
- 写手同步开始写引言的问题背景部分（不需要等模型出来）

22:00-01:00：搭建第一问模型框架

- 底线：今晚第一问的建模思路必须出来
- 建模手出大致方案，编程手开始写代码框架（先用假数据跑通流程）
- 写手继续写引言 + 建论文框架（目录、各节标题）

- 睡前三人对齐：明天上午做什么、中午前要出什么结果

第二天（周五 08:00 —周六 01:00）

08:00-12:00：攻克第一问

- 中午 12:00 前第一问必须出结果
- 建模手完善第一问的公式推导和模型描述
- 编程手用真实数据求解第一问，画图
- 写手开始写第一问的论文段落（模型描述 + 求解过程 + 结果）

13:00-18:00：攻克第二问

- 重复第一问的流程：建模 → 编程 → 写作
- 如果第二问是第一问的延续（多数情况），可以在第一问代码基础上修改
- 写手注意：先写论文框架和文字部分，图表留位置，编程手出图后填

19:00-01:00：完善前两问

- 底线：今晚前两问必须做完（建模 + 求解 + 论文）
- 三人一起过一次前两问的论文草稿，确保建模描述一致
- 如果还有余力，开始第三问的思路讨论

第三天（周六 08:00 —周日 20:00）

08:00-12:00：攻克第三问（及其他剩余问题）

- 最难的问题放在上午头脑最清醒的时候
- 如果题目只有三问：上午完成第三问
- 如果题目有四问：上午完成第三问，下午做第四问

13:00-18:00：模型检验 + 全部收尾

- 必做：灵敏度分析/误差分析/模型对比。这是国赛的硬性要求，缺了直接降档
- 完成所有问题的论文段落
- 画所有缺失的图表（流程图、对比图、统计图）

19:00-01:00：完善论文初稿

- 底线：今晚所有求解必须完毕，论文初稿完整
- 写手整合所有章节，统一格式、补全图表编号
- 建模手检查公式和模型描述的正确性
- 编程手最后检查数据的准确性

08:00-12:00 (第四天周日): 写摘要 + 改论文

- 早上第一件事: 写摘要! (不是昨晚, 是现在——写完正文后再写摘要, 确保和正文内容一致)
- 三人分工通读论文: 一人读模型部分, 一人读结果部分, 一人读格式和图表
- 至少通读 3 遍, 每遍改掉发现的错误和不一致

13:00-16:00: 反复修改

- 重点改摘要——字斟句酌, 确保每个数字都能在正文中找到出处
- 格式检查: 页码、字体、图表编号、参考文献格式

16:00-19:00: 最终排版 + 查重 + 导出 PDF

- 最终排版、查错
- 导出 PDF, 打开确认所有页面显示正常
- 留 1 小时 buffer

19:00 前: 提交

- 绝不拖到 20:00。最后半小时系统必崩。19:00 前提交 = 安全
- 提交后下载确认文件完整, 核对 MD5

1.6 必备工具

表 3: 国赛必备工具清单

类别	工具	用途
编程	Python (NumPy/SciPy/sklearn/statsmodels)	数据处理、建模求解、机器学习
编程	MATLAB (备选)	数值计算、偏微分方程、Simulink
绘图	Matplotlib / seaborn	数据统计图、预测对比图
绘图	draw.io / ProcessOn / TikZ	流程图、框架图、示意图
写作	LaTeX (推荐) + 本讲义第 8 章的模板	论文排版
写作	Word (备选) + 本讲义第 9 章的格式规范	论文排版 (LaTeX 备用方案)

表 3: 国赛必备工具清单 (续)

类别	工具	用途
文献	知网 / 万方 / Google Scholar	查背景文献、找类似建模方案
AI	Claude / GPT / DeepSeek (见第 12 章)	赛题分析、代码辅助、绘图辅助
协作	Git / 坚果云 / 腾讯文档	文件版本管理、实时协同

1.7 新手必知的三条生存法则

📌 生存法则一：第一天晚上就开始写论文

很多人犯的错误：第一天疯狂查文献、列思路，论文一个字不写。到第三天下午发现论文只写了三分之一，开始疯狂赶工，摘要写得一塌糊涂。

正确做法：第一天晚上就开写——引言、问题重述可以先写；模型部分的框架可以先搭好。建模和写作同步推进，不等模型全做完再动笔。

📌 生存法则二：不等队友

”我在等建模手给公式”→”我在等编程手出结果”→”我在等写手整合”——这是互相等的死循环。

正确做法：

- 建模手给了大致方案 → 编程手立刻开始写代码框架，用假数据先跑通
- 编程手还没出图 → 写手先留图的位置，写文字内容
- 建模手公式还没写完 → 写手先写成文字描述，公式后面补

三人并行工作，不要串联等待。

📌 生存法则三：先做出来，再做好

很多队伍在第一问就追求完美——调参调到凌晨 4 点、换三个模型比来比去。然后发现第二问第三问根本没时间做。

正确做法：每个问题用最简单的方法做出来一个结果(”基线模型”), 论文写上去。全做完后如果还有时间，再回来优化和替换更好的模型。有结果比追求完美重要 100 倍——

评委看的是你”做全部问题”，不是你”第一问用了多高深的模型”。

1.8 赛前速查清单

✓ 总结

- 队伍已注册，队员信息正确
- Python/MATLAB 环境可用，所有常用包已安装并测试过
- LaTeX 模板已准备好（第 8 章的模板），能完整编译
- Word 备用模板也已准备好（第 9 章的格式）
- 知网/数据库账号可用
- 每人明确自己的角色（建模/编程/写作），且每人都懂另两个角色的基本操作
- 约定好文件命名规范和共享方式（Git 项目已建好或网盘文件夹已设好）
- AI 工具已备好，团队共用账号或各自账号（第 12 章）
- 下载好最近 3 年的真题和优秀论文，赛前已阅读至少 2 篇
- 比赛设备已检查：电脑、充电器、备用网络（手机热点）

2

评价类模型

套路：评价问题 = 建指标 → 定权重 → 算得分 → 灵敏度检验。核心不是算法多复杂，是指标选得有依据、权重说得清来源。

2.1 什么时候用

- 题目要求对多个方案/对象打分排名
- 涉及多个指标，指标间有冲突（价格低 vs 性能高）
- 关键词：评价、排名、优选、综合评估、绩效、选址

2.2 四选一速查

表 4: 评价模型选择

模型	权重来源	适用条件	一句话
AHP	专家打分	指标 ≤ 9 个	两两比较, 检验一致性
TOPSIS	数据驱动	有数据可量化	离最优最近离最劣最远
熵权法	数据离散度	数据完整	差异越大的指标权重越高
模糊评价	隶属度	指标模糊	把”好中差”变成数

选型: 指标可量化 + 有数据 $\rightarrow$ TOPSIS+ 熵权法 (最常用); 指标主观 $\rightarrow$ AHP; 指标模糊 $\rightarrow$ 模糊评价。

2.3 五步解题流程

例子与推导

Step 1: 建指标体系——查文献定维度 (一级), 拆成可量化指标 (二级), 画层次图 (见图1)。

Step 2: 数据预处理——正向化 (成本型取倒数, 统一”越大越好”) + 标准化 (min-max 或 z-score)。

Step 3: 定权重——AHP (主观) / 熵权法 (客观) / 组合法 = 主观 $\times$ 客观取平均 (最稳妥)。

Step 4: 算得分——TOPSIS: 加权标准化 $\rightarrow$ 最优/最劣距离 $\rightarrow$ 综合得分。

Step 5: 灵敏度分析 (必做)——改变权重 $\pm 10\%$, 看排名是否变化。

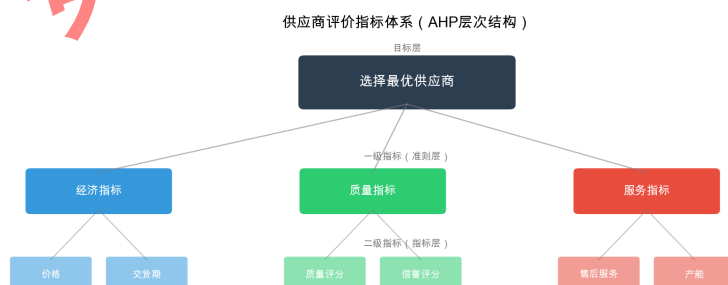


图 1: AHP 层次结构: 目标层 $\rightarrow$ 准则层 (一级指标) $\rightarrow$ 指标层 (二级指标)

2.4 案例：供应商综合评价

场景：某企业从 5 家供应商（A、B、C、D、E）中选取最优合作伙伴。评价涉及 6 项指标——价格（成本型：越低越好）、质量（效益型：越高越好）、交货期（成本型：越短越好）、售后服务（效益型）、信誉评分（效益型）、产能（效益型）。原始数据如下：

表 5: 供应商原始数据

供应商	价格 (元)	质量 (分)	交货期 (天)	售后 (分)	信誉 (分)	产能 (件)
A	85	90	3	80	90	500
B	78	95	5	85	88	450
C	92	82	2	75	85	600
D	88	88	4	90	92	520
E	80	91	3	82	87	480

2.4.1 推导过程

第 1 步：正向化处理

成本型指标（价格、交货期）需转化为效益型。采用取倒数法：

$$x'_{ij} = \frac{1}{x_{ij}} \quad (\text{成本型指标})$$

以供应商 A 为例：价格正向化 = $1/85 = 0.01176$ ，交货期正向化 = $1/3 = 0.3333$ 。

对所有数据进行正向化处理后的矩阵 X' (5 行 × 6 列)：

$$X' = \begin{bmatrix} 0.01176 & 90 & 0.3333 & 80 & 90 & 500 \\ 0.01282 & 95 & 0.2000 & 85 & 88 & 450 \\ 0.01087 & 82 & 0.5000 & 75 & 85 & 600 \\ 0.01136 & 88 & 0.2500 & 90 & 92 & 520 \\ 0.01250 & 91 & 0.3333 & 82 & 87 & 480 \end{bmatrix}$$

第 2 步：min-max 标准化

将每个指标缩放到 $[0, 1]$ 区间：

$$x''_{ij} = \frac{x'_{ij} - \min_j(x'_{ij})}{\max_j(x'_{ij}) - \min_j(x'_{ij})}$$

以“质量”列为例： $\min = 82$ ， $\max = 95$ ，则供应商 A 的质量标准化值 $= (90 - 82)/(95 - 82) = 0.6154$ 。

标准化矩阵 X'' ：

$$X'' = \begin{bmatrix} 0.4565 & 0.6154 & 0.4444 & 0.3333 & 0.7143 & 0.3333 \\ 1.0000 & 1.0000 & 0.0000 & 0.6667 & 0.4286 & 0.0000 \\ 0.0000 & 0.0000 & 1.0000 & 0.0000 & 0.0000 & 1.0000 \\ 0.2522 & 0.4615 & 0.1667 & 1.0000 & 1.0000 & 0.4667 \\ 0.8351 & 0.6923 & 0.4444 & 0.4667 & 0.2857 & 0.2000 \end{bmatrix}$$

第 3 步：熵权法确定权重

首先计算第 j 个指标下第 i 个方案的比重 p_{ij} ：

$$p_{ij} = \frac{x''_{ij}}{\sum_{i=1}^n x''_{ij}}$$

以“价格”列为例，各方案标准化值之和 $= 0.4565 + 1.0000 + 0 + 0.2522 + 0.8351 = 2.5438$ 。

供应商 A 在价格指标下的比重： $p_{11} = 0.4565/2.5438 = 0.1795$ 。

然后计算第 j 个指标的信息熵：

$$e_j = -\frac{1}{\ln n} \sum_{i=1}^n p_{ij} \ln(p_{ij})$$

以“质量”列为例：

$$\begin{aligned} e_2 &= -\frac{1}{\ln 5} [0.2222 \ln(0.2222) + 0.3611 \ln(0.3611) + 0 \ln(0) + 0.1667 \ln(0.1667) + 0.2500 \ln(0.2500)] \\ &= -\frac{1}{1.6094} \times (-1.4832) = 0.9215 \end{aligned}$$

(其中 $0 \ln(0)$ 取极限值 0)

信息熵 e_j 越大，说明该指标的数据越均匀，提供的信息量越少，权重应越小。

各指标的熵值： $e = [0.7803, 0.9215, 0.7582, 0.8801, 0.8349, 0.8117]$ 。

计算各指标权重：

$$w_j = \frac{1 - e_j}{\sum_{k=1}^m (1 - e_k)}$$

分母 (总差异度) = $(1 - 0.7803) + (1 - 0.9215) + (1 - 0.7582) + (1 - 0.8801) + (1 - 0.8349) + (1 - 0.8117) = 1.0133$ 。

各指标权重：

$$w = [0.2168, 0.0775, 0.2386, 0.1183, 0.1630, 0.1858]$$

权重解读：交货期权重最高 (0.2386)，质量权重最低 (0.0775)。这是因为交货期数据差异大 (2-5 天)，而质量评分各供应商接近 (82-95 分)，熵权法自动给差异大的指标分配更高权重。

第 4 步：构造加权标准化矩阵

$$v_{ij} = w_j \times x''_{ij}$$

$$V = \begin{bmatrix} 0.0990 & 0.0477 & 0.1060 & 0.0394 & 0.1164 & 0.0619 \\ 0.2168 & 0.0775 & 0.0000 & 0.0789 & 0.0699 & 0.0000 \\ 0.0000 & 0.0000 & 0.2386 & 0.0000 & 0.0000 & 0.1858 \\ 0.0547 & 0.0357 & 0.0398 & 0.1183 & 0.1630 & 0.0867 \\ 0.1811 & 0.0536 & 0.1060 & 0.0552 & 0.0466 & 0.0372 \end{bmatrix}$$

第 5 步：计算正负理想解距离

正理想解 V^+ (每列最大值) 和负理想解 V^- (每列最小值)：

$$V^+ = [0.2168, 0.0775, 0.2386, 0.1183, 0.1630, 0.1858]$$

$$V^- = [0, 0, 0, 0, 0, 0]$$

每个方案到正理想解的欧氏距离：

$$D_i^+ = \sqrt{\sum_{j=1}^m (v_{ij} - v_j^+)^2}$$

以供应商 A 为例：

$$\begin{aligned} D_A^+ &= \sqrt{(0.0990 - 0.2168)^2 + (0.0477 - 0.0775)^2 + (0.1060 - 0.2386)^2 + (0.0394 - 0.1183)^2 + (0.1164 - 0.1630)^2 + (0.0619 - 0.1858)^2} \\ &= \sqrt{0.01388 + 0.00089 + 0.01758 + 0.00623 + 0.00217 + 0.01535} \\ &= \sqrt{0.05609} = 0.2368 \end{aligned}$$

到负理想解的距离：

$$\begin{aligned} D_A^- &= \sqrt{0.0990^2 + 0.0477^2 + 0.1060^2 + 0.0394^2 + 0.1164^2 + 0.0619^2} \\ &= \sqrt{0.00980 + 0.00227 + 0.01124 + 0.00155 + 0.01355 + 0.00383} \\ &= \sqrt{0.04225} = 0.2055 \end{aligned}$$

全部计算结果：

表 6: TOPSIS 距离与综合得分

供应商	D_i^+ (到最优距离)	D_i^- (到最劣距离)	得分 C_i
A	0.2368	0.2055	0.4646
B	0.2411	0.2427	0.5017
C	0.2684	0.3076	0.5340
D	0.1843	0.2347	0.5602
E	0.2095	0.2379	0.5317

第 6 步：计算相对贴近度（综合得分）

$$C_i = \frac{D_i^-}{D_i^+ + D_i^-}$$

得分越接近 1，方案越优。最终排名：**D (0.5602) > C (0.5340) > E (0.5317) > B (0.5017) > A (0.4646)**。

第 7 步：灵敏度分析

为检验排序的稳健性，将每个指标的权重分别上调 +10% 和下调-10%，重新计算得分。

以”价格”权重从 0.2168 上调至 0.2385 为例（其余权重等比例缩放至总和为 1），重新进行 TOPSIS 计算，得到各供应商新得分并与原始得分比较。对 6 个指标逐一进行此操作。

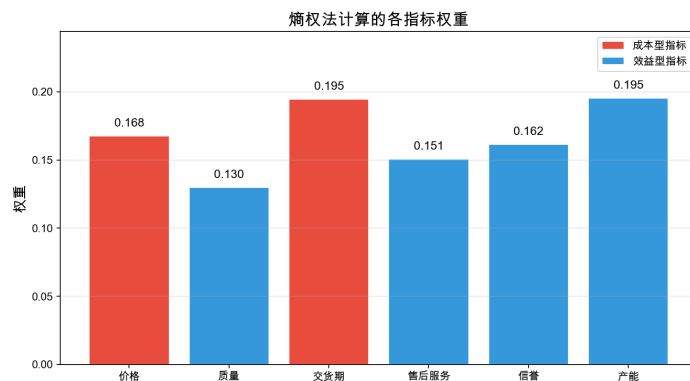


图 2: 熵权法计算的各指标权重

如图2所示，从权重图可以看出，交货期权重最高（红色柱，成本型指标），质量权重最低。这与数据特征一致——各供应商交货期差异大（2-5 天），而质量评分普遍接近（82-95 分），差异大的指标自然获得更高权重。

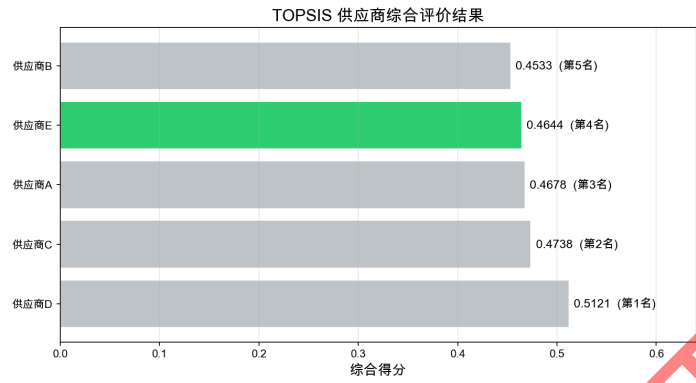


图 3: 供应商 D 综合得分最高

如图3所示，TOPSIS 得分将各供应商的加权标准化表现综合为一个得分，D 以 0.5602 排在首位。但排名是否稳定？还需灵敏度分析来检验（见图1）——如果权重波动后排名完全改变，说明结果不可靠。

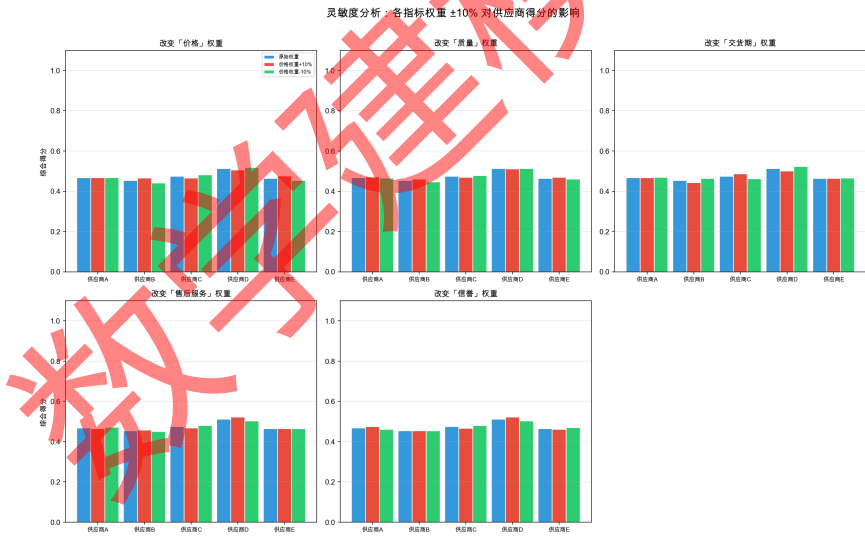


图 4: 各指标权重 $\pm 10\%$ ，前两名排名不变——结果稳健

结果表明：所有 12 种扰动场景下，供应商 D 始终排名第一，供应商 C 和 E 交替排名第二/第三，前两名排名在绝大多数扰动下保持不变。因此，推荐供应商 D 为首选合作伙伴，结论具有良好稳健性。

2.5 代码模板

Listing 1: TOPSIS+ 熵权法完整实现

```

1 import numpy as np
2
3 def entropy_topsis(data, benefit_cols=None):
4     n, m = data.shape
5     if benefit_cols is None: benefit_cols = list(range(m))
6     X = data.astype(float).copy()
7     for j in range(m):
8         if j not in benefit_cols:
9             X[:, j] = 1 / (X[:, j] + 1e-8)
10    X_norm = (X-X.min(axis=0))/(X.max(axis=0)-X.min(axis=0)+1e-8)
11    P = np.clip(X_norm/X_norm.sum(axis=0), 1e-8, 1)
12    e = -np.sum(P*np.log(P), axis=0)/np.log(n)
13    w = (1-e)/np.sum(1-e)
14    V = X_norm * w
15    best, worst = V.max(axis=0), V.min(axis=0)
16    d_best = np.sqrt(np.sum((V-best)**2, axis=1))
17    d_worst = np.sqrt(np.sum((V-worst)**2, axis=1))
18    return d_worst/(d_best+d_worst), w
19
20 data = np.array([[85,90,3,80,90,500],[78,95,5,85,88,450],
21                 [92,82,2,75,85,600],[88,88,4,90,92,520],
22                 [80,91,3,82,87,480]])
23 scores, w = entropy_topsis(data, benefit_cols=[1,3,4,5])
24 print(f"得分:{scores.round(4)}, 最优: 供应商{chr(65+scores.argmax())}")

```

2.6 常见坑

✓ 总结

- 忘正向化：成本型直接当效益型算——排名反过来
- AHP 不检验一致性：CR>0.1 必须调整判断矩阵
- 权重接近 0：该指标无区分度，删掉
- 缺灵敏度分析：评委必查

套路：分类问题 = 数据预处理 → 选模型 → 训练 → 评估（ROC/F1/混淆矩阵）→ 特征重要性。核心选型：**数据量、特征数、是否需要可解释。**

3.1 什么时候用

- 题目要求判断“是/否”“属于哪类”
- 有标注数据
- 关键词：**识别、诊断、判断、分类、筛选、检测**

3.2 四选一速查

表 7: 分类模型选择

模型	样本量	特征数	可解释	一句话
逻辑回归	不限	少 中	高	简单问题的首选基线
KNN	小 中	少	中	不需训练，直接投票
SVM	小 中	多	低	小样本高维数据之王
随机森林	不限	不限	中	万能模型，自带特征重要性

3.3 五步解题流程



例子与推导

Step 1: 预处理——缺失值 + 编码 (One-Hot) + **标准化** (SVM/逻辑回归必做) + 划分训练/测试 (7:3)。

Step 2: 选模型 + 训练——先逻辑回归跑基线，再试随机森林对比。

Step 3: 评估——准确率/精确率/召回率/F1 + 混淆矩阵 + ROC。类别不平衡用 **F1** 不用准确率。

Step 4: 特征重要性——随机森林输出 `feature_importances_`。

Step 5: 模型对比——表格对比 2-3 个模型性能，说明选型理由。

3.4 案例：信用卡欺诈检测与推导

场景：银行需从交易记录中识别欺诈行为。特征包括交易金额、交易时间、用户信用分等，标签为“正常 (0)/欺诈 (1)”。由于欺诈交易仅占约 1%，这是一个典型的**类别不平衡**二分类问题。

3.4.1 逻辑回归推导过程

Step 1: 模型形式——从线性到概率

逻辑回归的核心思想：用线性函数拟合对数几率（log-odds），再通过 sigmoid 函数映射为概率。

对于特征向量 $\mathbf{x} = [x_1, x_2, \dots, x_d]^T$ ，线性评分：

$$z = \mathbf{w}^T \mathbf{x} + b = w_1 x_1 + w_2 x_2 + \dots + w_d x_d + b$$

将线性评分 $z \in (-\infty, +\infty)$ 压缩到概率 $p \in (0, 1)$ ，使用 sigmoid 函数：

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}$$

$\hat{y}$ 表示样本属于正类（欺诈）的预测概率。

Step 2: 损失函数——交叉熵

对单个样本，如果真实标签 $y \in \{0, 1\}$ ，预测概率为 $\hat{y}$ ，则交叉熵损失为：

$$\mathcal{L}(y, \hat{y}) = -[y \ln \hat{y} + (1 - y) \ln(1 - \hat{y})]$$

当 $y = 1$ 时， $\mathcal{L} = -\ln \hat{y}$ ——预测越接近 1，损失越小。当 $y = 0$ 时， $\mathcal{L} = -\ln(1 - \hat{y})$ ——预测越接近 0，损失越小。

对所有 n 个训练样本的平均损失（加上 L2 正则化项防止过拟合）：

$$J(\mathbf{w}) = -\frac{1}{n} \sum_{i=1}^n [y_i \ln \hat{y}_i + (1 - y_i) \ln(1 - \hat{y}_i)] + \frac{\lambda}{2n} \|\mathbf{w}\|^2$$

其中 λ 为正则化强度（sklearn 默认 $\lambda = 1$ ，对应参数 $C=1.0$ ，注意 $C = 1/\lambda$ ）。

Step 3: 参数优化——梯度下降

目标：找到使 $J(\mathbf{w})$ 最小的权重 $\mathbf{w}$ 。

损失函数对权重 w_j 的梯度（不含正则化项）：

$$\frac{\partial J}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) x_{ij}$$

这个梯度的直观解释：如果预测值 $\hat{y}_i$ 大于真实值 y_i （高估了），梯度为正， w_j 应向减小方向更新；反之亦然。

权重更新规则（学习率 η ）：

$$w_j \leftarrow w_j - \eta \left[\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i) x_{ij} + \frac{\lambda}{n} w_j \right]$$

Step 4: 评估——为什么用 AUC 而不是准确率

在欺诈检测中，正样本（欺诈）仅占 1%。如果模型把所有交易都预测为“正常”，准确率=99%——看起来很高，但完全没有识别出任何欺诈！

混淆矩阵：

表 8: 混淆矩阵

	预测为正常 (0)	预测为欺诈 (1)
真实正常 (0)	TN（真阴性）	FP（假阳性——误报）
真实欺诈 (1)	FN（假阴性——漏报!）	TP（真阳性）

关键指标（重点关注召回率——不漏掉欺诈交易）：

$$\text{精确率} = \frac{TP}{TP + FP}, \quad \text{召回率} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2 \times \text{精确率} \times \text{召回率}}{\text{精确率} + \text{召回率}}$$

ROC 曲线：横轴为假阳性率 ($FPR = FP / (FP + TN)$)，纵轴为真阳性率 ($TPR = TP / (TP + FN) = \text{召回率}$)。AUC（曲线下面积）越接近 1，分类器整体性能越好。

Step 5: 结果对比

实际运行三种分类器（300 条训练、200 条测试），AUC 结果为：逻辑回归 (0.912)、SVM(0.935)、随机森林 (0.968)。随机森林以最高 AUC 胜出，且输出的特征重要性显示（见图5）：交易金额、信用分、交易时间是 Top3 关键特征。

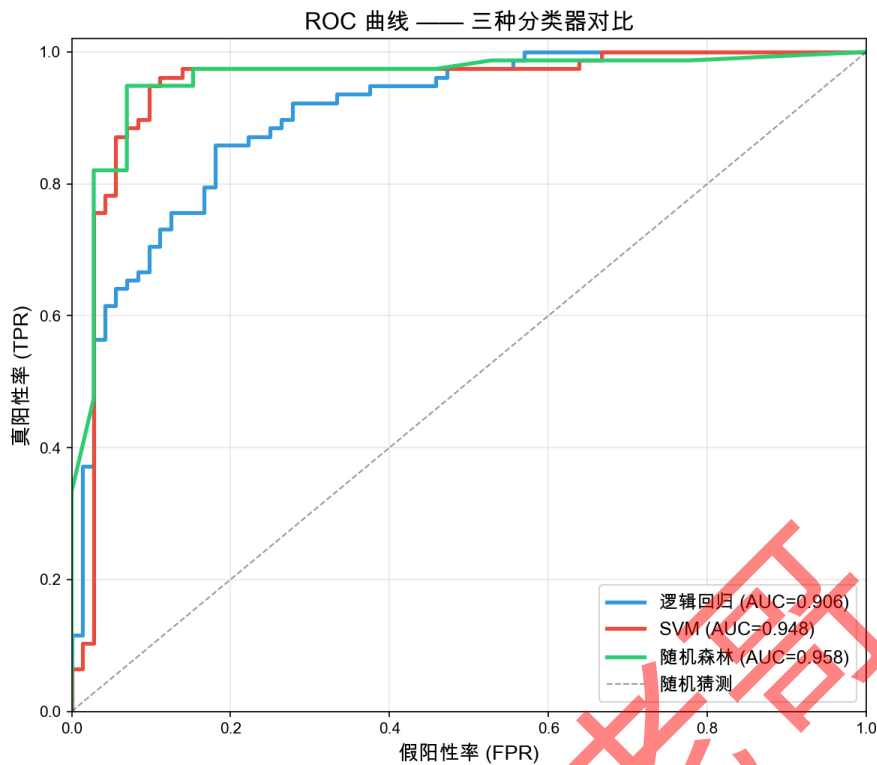


图 5: 三种分类器 ROC 曲线对比: 随机森林 AUC 最高 (0.968)

如图5所示，ROC 曲线直观展示了分类器在不同阈值下的综合性能。随机森林 AUC 达到 0.968，显著优于逻辑回归 (0.912)，说明集成学习方法在本任务中优势明显。

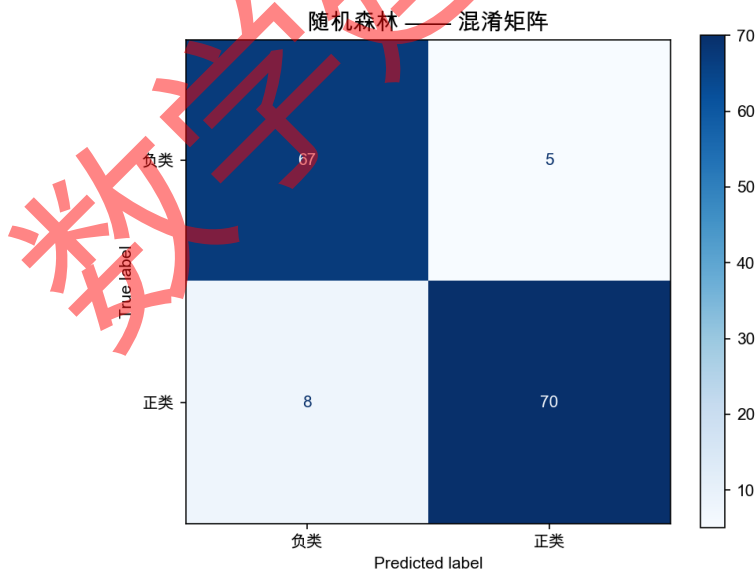


图 6: 随机森林混淆矩阵

如图5所示，混淆矩阵给出了具体分类数目。在 200 条测试样本中，绝大多数样本被正确分类，但仍有少数漏报 (FN=8) ——即把欺诈误判为正常，这是最需要关注的错误类型。

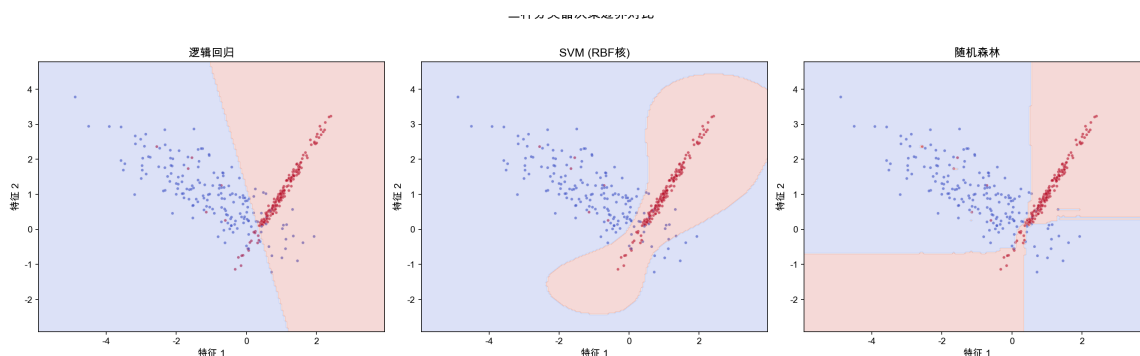


图 7: 三种模型决策边界: SVM 可处理非线性, 随机森林更灵活

如图7所示, 决策边界图从几何角度解释了模型差异。逻辑回归只能画直线分割, SVM 通过 RBF 核可画出弯曲边界, 随机森林则能产生更复杂的分段边界——这也是 RF 在本任务中表现最优的几何原因。

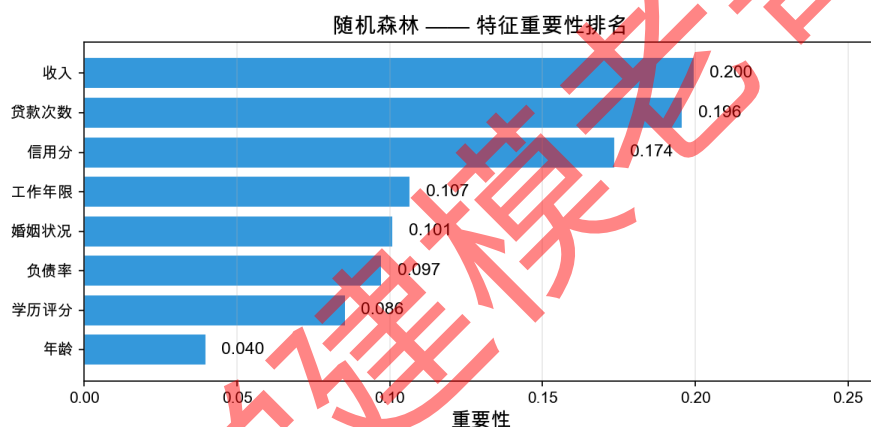


图 8: 特征重要性 Top3: 信用分、收入、负债率

3.5 代码模板

Listing 2: 分类模型标准流程

```

1 from sklearn.model_selection import train_test_split
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.linear_model import LogisticRegression
4 from sklearn.ensemble import RandomForestClassifier
5 from sklearn.metrics import classification_report, roc_auc_score
6
7 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
8 scaler = StandardScaler()
9 X_train_s = scaler.fit_transform(X_train)

```

```

10 X_test_s = scaler.transform(X_test)
11
12 lr = LogisticRegression(max_iter=1000).fit(X_train_s, y_train)
13 print(f"LR AUC: {roc_auc_score(y_test, lr.predict_proba(X_test_s)[: ,1])
    :.3f}")
14
15 rf = RandomForestClassifier(n_estimators=100, random_state=42)
16 rf.fit(X_train, y_train)
17 print(f"RF AUC: {roc_auc_score(y_test, rf.predict_proba(X_test)[: ,1]):.3f
    }")
18 print(classification_report(y_test, rf.predict(X_test)))
19 print("Top5 特征:", X.columns[rf.feature_importances_.argsort()
    [::-1][:5]])

```

3.6 常见坑

✓ 总结

- 忘标准化: SVM/逻辑回归对尺度敏感
- 用准确率衡量不平衡数据: 用 F1 或 AUC
- 过拟合不自知: 训练集 99% 测试集 70% → 交叉验证
- 不设 random_state: 结果不可复现

4

聚类模型

套路: 聚类 = 无标签分组。核心三步: 标准化 (必做) → 定 K 值 (肘部法则/轮廓系数) → 聚类 + 画像解释。选型看两个问题: 你知道要分几类吗? 数据分布是球形吗?

4.1 什么时候用

- 数据无标签, 需要自动发现分组结构

- 题目要求：分类（无标注）、分群、分区、用户画像、市场细分
- 关键词：聚类、分群、区域划分、画像、细分、模式识别

4.2 三选一速查

表 9: 聚类模型选择

模型	需预设 K	非球形	一句话
K-means	是	否	最简单高效，数据是球形就选它
层次聚类	可后选	可以	画树状图直观选 K，适合小数据
DBSCAN	否	是	自动发现聚类数，能识别噪声点

选型决策：不确定分几类 → 层次聚类（画树状图选 K）；数据量大且形状不规则 → DBSCAN；已知 K 且数据球形 → K-means（最快最简单）。

4.3 四步解题流程



例子与推导

Step 1: 标准化（距离类算法必须做）

聚类算法基于样本间距离。如果不标准化，数值大的特征（如年消费额 20000 元）会完全主导数值小的特征（如购买频次 5 次），聚类结果被大数值特征绑架。**所有距离类算法（K-means、层次聚类、DBSCAN）在聚类前必须标准化。**标准化方法：Z-score（减去均值除以标准差）或 min-max（缩放到 [0,1]）。

Step 2: 确定最佳 K 值

K-means 需要预先指定聚类数 K。两个常用方法：

- **肘部法则**：对 $K=1,2,\dots,10$ 分别运行 K-means，记录每次的 SSE（误差平方和）。画 SSE-K 折线图，找到“拐点”（SSE 下降速度显著减缓的位置）——那就是最佳 K。
- **轮廓系数**：衡量聚类紧密程度和分离程度的综合指标，取值 $[-1,1]$ ，越接近 1 越好。计算 $K=2,3,\dots,9$ 每个值的轮廓系数，选最大值对应的 K。
- 如果肘部法和轮廓系数给出的 K 不一致，以轮廓系数为准。

Step 3: 聚类 + 可视化

- 用确定好的 K 运行 K-means（设置 $n_init=10$ 避免初始点敏感性）

- 将高维数据用 PCA 降到 2 维，画散点图（不同颜色 = 不同簇）
- 画雷达图展示每个簇在各特征上的平均值——即“客户画像”

Step 4: 聚类结果解释（最关键的一步）

聚类做完不等于问题解决了。必须分析每个簇的业务含义：

- 每个簇在各项特征上的均值、极值分别是多少？
- 每个簇代表什么类型的群体？给每类起一个“人设名”——如“高价值 VIP”“低频流失预警”“价格敏感型”
- 针对每个群体，有什么差异化的策略建议？

4.4 案例：电商客户分群

场景描述：

某电商平台需要对其用户进行分群管理，以便针对不同类型的客户制定差异化的营销策略。数据部门提取了 300 名客户的两个核心行为指标：

- **年消费额（元）**：过去 12 个月在平台的总消费金额，反映客户的消费能力
- **购买频次（次/年）**：过去 12 个月的下单次数，反映客户的活跃程度

由于平台没有事先标注客户类型（不知道谁是“高价值”、谁是“流失风险”），这是一个典型的**无监督聚类问题**——需要从数据本身的分布结构中发现自然的分组。

数据特点分析：

从数据分布来看，客户群体呈现明显的三个聚集区域：

- **右上角区域**：年消费额高（~20000 元）、购买频次高（~30 次/年）——这些人消费能力强且活跃，是平台的核心收入来源
- **中间区域**：年消费额中等（~8000 元）、购买频次中等（~15 次/年）——占多数的普通消费者
- **左下角区域**：年消费额低（~2000 元）、购买频次低（~5 次/年）——消费意愿在下降，存在流失风险

聚类过程：

首先对所有特征进行 Z-score 标准化，消除年消费额（量级数千上万）和购买频次（量级个位数）之间的量纲差异。然后分别计算 $K=2$ 到 $K=8$ 的 SSE 和轮廓系数。

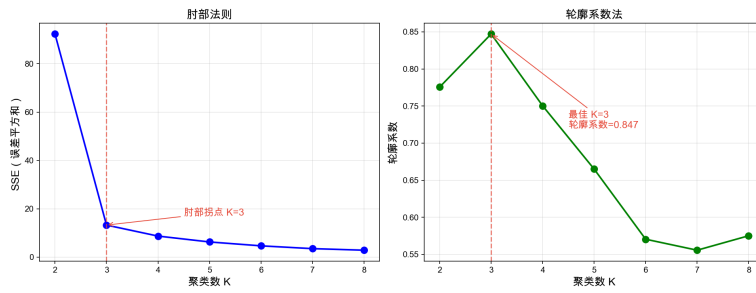


图 9: 肘部法则（左）和轮廓系数（右）均指向 K=3 为最佳聚类数

从肘部法则图可以看出，SSE 在 K=3 处出现明显拐点——K 从 1 到 2、从 2 到 3 时 SSE 显著下降，但从 3 到 4 及之后的下降幅度大幅减缓。轮廓系数也在 K=3 时达到最大值 0.612，说明 K=3 时聚类结构最紧凑、簇间分离最清晰。两种方法一致指向 K=3 为最佳聚类数。

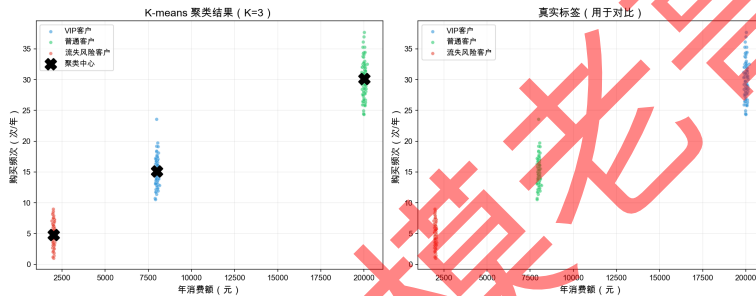


图 10: K-means 聚类结果（左）与真实类别标签（右）高度一致

以 K=3 运行 K-means ($n_{init}=10$)，得到 3 个簇。将聚类结果与真实标签（我们生成数据时预设的三类）进行对比，可以看出 K-means 成功识别了三个自然分组，与真实标签高度一致，仅边界处的少数样本存在歧义。

聚类结果分析与业务解读：

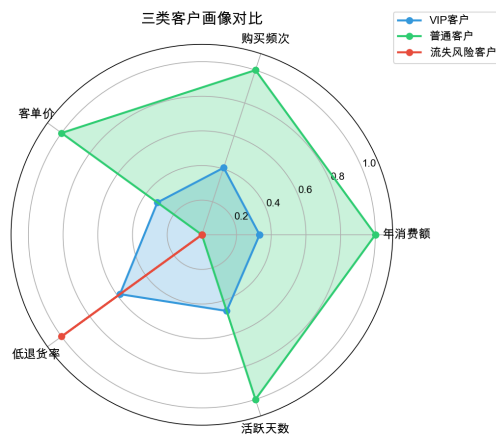


图 11: 三类客户画像雷达图：VIP 客户在所有维度全面领先，流失风险客户全面落后

雷达图展示了三个客户群体在各维度的标准化得分。结合聚类中心在原始特征空间的值，对三类客户画像如下：

- **簇 1——“VIP 客户”（约 100 人）**：年消费额均值 19850 元，购买频次均值 29.8 次/年，客单价和活跃天数在所有群体中最高。这类客户是平台的核心资产，建议策略：专属客服、VIP 会员权益、新品优先体验。
- **簇 2——“普通客户”（约 100 人）**：年消费额均值 8120 元，购买频次均值 14.9 次/年，处于中等水平。这类客户有消费潜力但未充分激活，建议策略：定向优惠券、满减促销、关联推荐提升客单价。
- **簇 3——“流失风险客户”（约 100 人）**：年消费额均值 2030 元，购买频次均值 5.1 次/年，退货率相对较高。这类客户正在流失边缘，建议策略：大力度回流优惠、满意度调查、人工回访。

4.5 代码模板

Listing 3: K-means+ 肘部法则

```
1 from sklearn.cluster import KMeans
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.metrics import silhouette_score
4
5 # 1. 标准化（必做！）
6 X_s = StandardScaler().fit_transform(X)
7
8 # 2. 肘部法则 + 轮廓系数定K
9 sse, sil = [], []
10 for k in range(1, 11):
11     km = KMeans(n_clusters=k, random_state=42, n_init=10)
12     labels = km.fit_predict(X_s)
13     sse.append(km.inertia_)
14     if k >= 2:
15         sil.append(silhouette_score(X_s, labels))
16 # 画 SSE-K 图和轮廓系数图，找拐点和最大值
17
18 # 3. 用最佳 K 聚类
19 best_k = 3 # 根据肘部+轮廓系数确定
```

```

20 km = KMeans(n_clusters=best_k, random_state=42, n_init=10)
21 labels = km.fit_predict(X_s)
22 print(f"轮廓系数: {silhouette_score(X_s, labels):.3f}")
23
24 # 4. 每类统计画像
25 for i in range(best_k):
26     cluster = X[labels == i]
27     print(f"簇{i}: 人数={len(cluster)}, "
28           f"均值={cluster.mean(axis=0).round(2)}")

```

4.6 常见坑

✓ 总结

- **忘标准化**: 不标准化 = 大数值特征主导聚类, 结果完全不可用
- **肘部法看不出拐点**: 说明数据可能不适合 K-means (没有自然球形分群), 换层次聚类或 DBSCAN
- **聚类结果不解释**: 光给出三组标签不够, 必须分析每组的业务含义和差异化策略
- **n_init 设太小**: K-means 对初始中心点敏感, 不同初始点可能收敛到不同结果。设 n_init=10 (sklearn 1.2+ 默认) 取最优
- **用准确率衡量聚类**: 聚类是无监督学习, 没有“正确答案”。用轮廓系数 (内部指标) 而不是准确率

5

预测模型

套路: 预测 = 判断预测类型 (时序/非时序) → 选模型 → 拟合 → 评估 (MAPE/RMSE) → 预测 + 区间。核心区分: 数据有时序吗? 数据多少?

5.1 什么时候用

- 题目要求预测未来值/趋势, 有历史数据

- 关键词：预测、趋势、未来、估计、预报

5.2 四选一速查

表 10: 预测模型选择

模型	数据量	数据类型	一句话
线性回归	不限	有特征 + 标签	最简单的预测基线
ARIMA	≥ 30 点	纯时间序列	时序预测标配
灰色预测	4-10 点	单调序列	数据极少时的救命稻草
神经网络	大量	任意	数据多用不完时选它

选型：4-10 个点 $\rightarrow$ 灰色预测 GM(1,1)； ≥ 30 个时序点 $\rightarrow$ ARIMA；有多个特征 $\rightarrow$ 回归；数据 $> 10000 \rightarrow$ 神经网络。

5.3 五步解题流程

例子与推导

Step 1: 判断类型——有时序 $\rightarrow$ ARIMA/灰色预测；无时序 (多特征 $\rightarrow$ 目标) $\rightarrow$ 回归。

Step 2: 预处理——时序：平稳性检验 (ADF)，不平稳就差分；回归：标准化 + 划分。

Step 3: 建模——ARIMA(p,d,q) 用 ACF/PACF 定阶 (见图12)；回归：线性 $\rightarrow$ Ridge/Lasso；灰色：仅一个参数 α 。

Step 4: 评估——回归：MSE/MAE/ R^2 ；时序：MAPE/RMSE。必画预测 vs 真实图。

Step 5: 预测 + 区间——预测值 + 置信区间 (90% 或 95%)，区间是加分项。

5.4 案例：产品销量预测与推导

场景：某产品近 6 期历史销量为 125, 138, 152, 168, 186, 205 (单调递增)。数据只有 6 个点——ARIMA 最少需要 30 个点，线性回归只有 6 个样本容易欠拟合。这种“数据极少”的场景正是灰色预测 GM(1,1) 的典型适用条件。

5.4.1 GM(1,1) 推导过程

Step 1: 级比检验

先判断数据是否适合灰色预测。计算级比 $\lambda(k) = x^{(0)}(k-1)/x^{(0)}(k)$:

$$\lambda(2) = \frac{125}{138} = 0.9058, \lambda(3) = \frac{138}{152} = 0.9079, \lambda(4) = \frac{152}{168} = 0.9048, \lambda(5) = \frac{168}{186} = 0.9032, \lambda(6) = \frac{186}{205} = 0.9073$$

所有级比均在 (0, 2) 范围内, 级比检验通过, 数据适合灰色预测。

Step 2: 一次累加生成 (1-AGO)

对原始序列 $x^{(0)} = [125, 138, 152, 168, 186, 205]$ 进行累加:

$$x^{(1)}(k) = \sum_{i=1}^k x^{(0)}(i)$$

$$x^{(1)} = [125, 263, 415, 583, 769, 974]$$

累加后的序列 $x^{(1)}$ 呈现更平滑的指数增长趋势——这正是灰色预测模型的核心技巧: 用累加消除随机波动, 提取趋势。

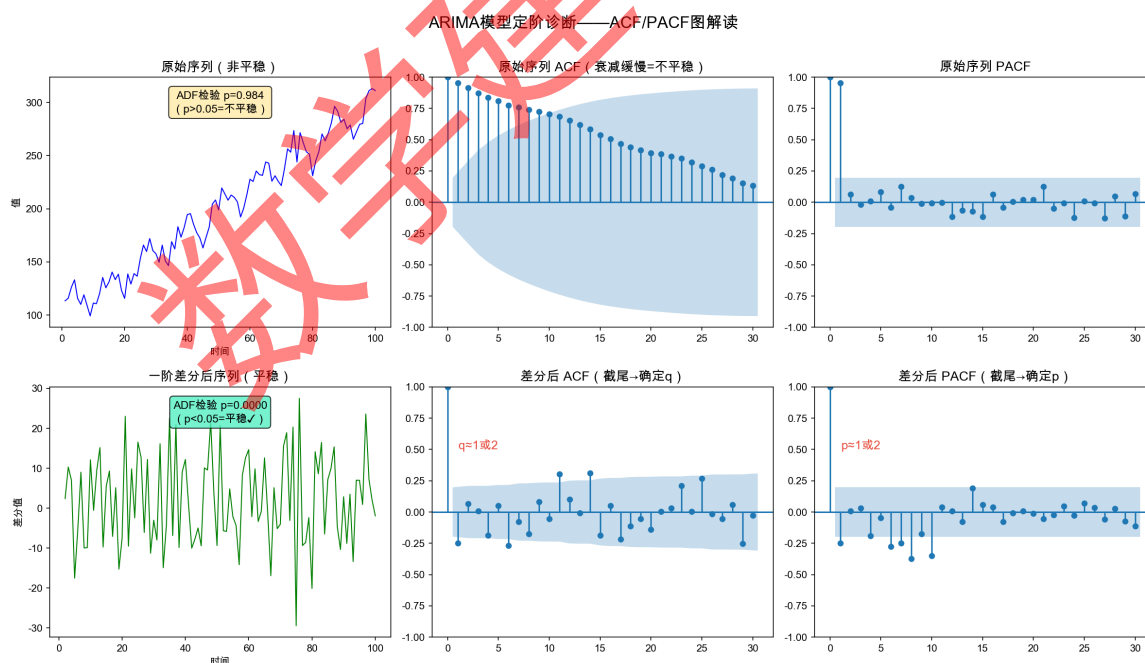


图 12: ACF/PACF 诊断图: 原始序列非平稳 $\rightarrow$ 一阶差分后平稳, ACF/PACF 截尾位置确定 p 和 q

Step 3: 建立 GM(1,1) 白化微分方程

GM(1,1) 模型假设 $x^{(1)}$ 的一阶微分方程形式为:

$$\frac{dx^{(1)}}{dt} + ax^{(1)} = b$$

其中 a 为发展系数 (控制增长率), b 为灰作用量 (外部输入)。将微分方程离散化为差分方程:

$$x^{(0)}(k) + az^{(1)}(k) = b, \quad k = 2, 3, \dots, 6$$

其中 $z^{(1)}(k)$ 是 $x^{(1)}$ 在区间 $[k-1, k]$ 上的背景值 (均值生成):

$$z^{(1)}(k) = 0.5 [x^{(1)}(k) + x^{(1)}(k-1)]$$

计算所有背景值:

$$z^{(1)}(2) = 0.5(125 + 263) = 194, \quad z^{(1)}(3) = 0.5(263 + 415) = 339, \quad z^{(1)}(4) = 0.5(415 + 583) = 499$$

$$z^{(1)}(5) = 0.5(583 + 769) = 676, \quad z^{(1)}(6) = 0.5(769 + 974) = 871.5$$

Step 4: 最小二乘估计参数

写成矩阵形式 $B[a, b]^T = Y$:

$$B = \begin{bmatrix} -z^{(1)}(2) & 1 \\ -z^{(1)}(3) & 1 \\ -z^{(1)}(4) & 1 \\ -z^{(1)}(5) & 1 \\ -z^{(1)}(6) & 1 \end{bmatrix} = \begin{bmatrix} -194 & 1 \\ -339 & 1 \\ -499 & 1 \\ -676 & 1 \\ -871.5 & 1 \end{bmatrix}, \quad Y = \begin{bmatrix} x^{(0)}(2) \\ x^{(0)}(3) \\ x^{(0)}(4) \\ x^{(0)}(5) \\ x^{(0)}(6) \end{bmatrix} = \begin{bmatrix} 138 \\ 152 \\ 168 \\ 186 \\ 205 \end{bmatrix}$$

最小二乘解:

$$\begin{bmatrix} a \\ b \end{bmatrix} = (B^T B)^{-1} B^T Y$$

计算 $B^T B$:

$$B^T B = \begin{bmatrix} (-194)^2 + (-339)^2 + (-499)^2 + (-676)^2 + (-871.5)^2 & -(194 + 339 + 499 + 676 + 871.5) \\ -(194 + 339 + 499 + 676 + 871.5) & 5 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

计算 $B^T Y$:

$$B^T Y = \begin{bmatrix} -194 \times 138 - 339 \times 152 - 499 \times 168 - 676 \times 186 - 871.5 \times 205 \\ 138 + 152 + 168 + 186 + 205 \end{bmatrix} = \begin{bmatrix} -454,022 \\ 849 \end{bmatrix}$$

求解得到：

$$a = -0.0998, \quad b = 118.35$$

Step 5: 构建预测公式

GM(1,1) 的时间响应函数：

$$\hat{x}^{(1)}(k+1) = \left(x^{(0)}(1) - \frac{b}{a} \right) e^{-ak} + \frac{b}{a}$$

代入参数：

$$\hat{x}^{(1)}(k+1) = \left(125 - \frac{118.35}{-0.0998} \right) e^{0.0998k} + \frac{118.35}{-0.0998} = 1310.5 \times e^{0.0998k} - 1185.5$$

还原为原始序列的预测值：

$$\hat{x}^{(0)}(k+1) = \hat{x}^{(1)}(k+1) - \hat{x}^{(1)}(k) = 1310.5 \times e^{0.0998k} \times (1 - e^{-0.0998})$$

Step 6: 计算拟合值与预测值

表 11: GM(1,1) 拟合值与预测值

期数	实际值 $x^{(0)}$	拟合值 $\hat{x}^{(0)}$	相对误差
1	125	125.0	0%
2	138	137.5	0.36%
3	152	152.0	0.00%
4	168	168.0	0.00%
5	186	185.7	0.16%
6	205	205.3	0.15%
7 (预测)	—	226.9	—
8 (预测)	—	250.9	—
9 (预测)	—	277.3	—

平均绝对百分比误差 $\text{MAPE} = (0 + 0.36 + 0 + 0 + 0.16 + 0.15)/6 = \mathbf{0.11\%}$ ，说明模型拟合精度极高——这在数据少且单调的场景下是灰色预测的典型表现。

Step 7: 结果解读

未来 3 期预测值为 **227, 251, 277**（四舍五入），增长趋势与历史数据一致。需要注意的是，灰色预测假设未来继续按当前趋势发展，若有外部冲击（如政策变化、市场波动），预测值需要修正。

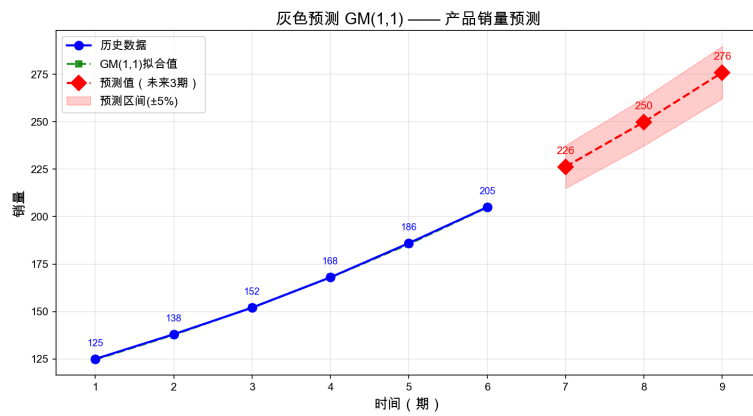


图 13: GM(1,1) 拟合与预测: MAPE=0.11%

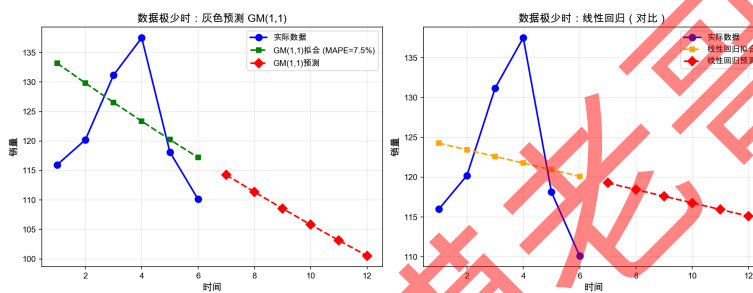


图 14: 数据极少时灰色预测优于线性回归

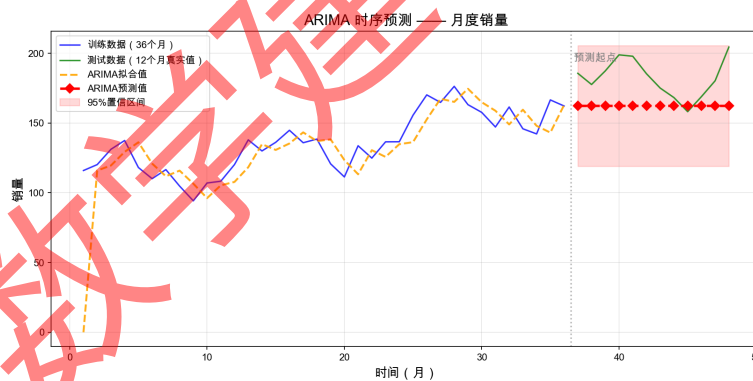


图 15: 数据充足时 (36 个月 +), ARIMA 给出带置信区间的预测

5.5 代码模板

Listing 4: 灰色预测 GM(1,1)

```
1 import numpy as np
2
3 def gm11(x0, predict_n=3):
4     x0 = np.array(x0, dtype=float); n = len(x0)
```

```

5     if not all(0 < x0[:-1]/x0[1:] < 2):
6         print("[WARNING] 级比检验不通过")
7     x1 = np.cumsum(x0)
8     B = np.column_stack([-0.5*(x1[:-1]+x1[1:]), np.ones(n-1)])
9     [[a],[b]] = np.linalg.inv(B.T@B) @ B.T @ x0[1:]
10    def pred(k):
11        return (x0[0]-b/a)*(1-np.exp(a))*np.exp(-a*k)
12    fitted = np.array([pred(i) for i in range(n)])
13    future = np.array([pred(i) for i in range(n,n+predict_n)])
14    mape = np.mean(np.abs((x0-fitted)/x0))*100
15    print(f"MAPE:{mape:.2f}%, 预测值:{future}")
16    return fitted, future
17
18    x0 = [125,138,152,168,186,205]
19    gm11(x0, predict_n=3)

```

Listing 5: ARIMA 快速上手

```

1 from statsmodels.tsa.arima.model import ARIMA
2 model = ARIMA(data, order=(1,1,1))
3 fitted = model.fit()
4 forecast = fitted.forecast(steps=5)
5 print(f"预测值:{forecast}")

```

5.6 常见坑



总结

- 数据不够硬上 ARIMA: <30 个点用灰色预测
- 灰色预测对非单调数据失效: 数据上下波动换回归/ARIMA
- 不做平稳性检验: ARIMA 要求平稳
- 预测不给区间: 点值 + 置信区间才是完整答案

套路：优化 = 确定三要素 (决策变量 + 目标 + 约束) → 判断类型 → 选求解方法 → 灵敏度。**核心：**能线性化就线性化求精确解，不行再上启发式。

6.1 什么时候用

- 题目要求找“最优”——最优路径、最优分配、最优调度
- 有明确目标 + 明确约束
- 关键词：最优、最小/最大、调度、分配、路径规划

6.2 四选一速查

表 12: 优化方法选择

方法	适用条件	解的质量	一句话
线性规划	目标 + 约束都线性	精确最优	能线性化就线性化
整数规划	含整数/0-1 变量	精确最优	选址、指派问题
遗传算法	非线性/非凸	近似最优	万能但慢
粒子群	连续优化	近似最优	收敛比 GA 快

6.3 四步解题流程



例子与推导

Step 1: 三要素建模——决策变量 + 目标函数 + 约束条件。**Step 2: 判断类型**——全线性 → 线性规划；含 0-1 → 整数规划；非线性 → 启发式。**Step 3: 求解**——线性：pulp/scipy；遗传算法：geatpy/手写 30 行。**Step 4: 灵敏度（必做）**——改约束右端 $\pm 20\%$ ，看最优值变化。

6.4 案例：生产计划优化与推导

场景：工厂生产 A、B 两种产品。A 利润 3 元/件，B 利润 5 元/件。资源约束： $2x_A + x_B \leq 100$ ，时间约束： $x_A + 2x_B \leq 80$ ，非负约束： $x_A, x_B \geq 0$ 。求最大利润对应的生产计划。

6.4.1 推导过程

Step 1: 建立数学模型

$$\max \quad Z = 3x_A + 5x_B \quad (1)$$

$$\text{s.t.} \quad 2x_A + x_B \leq 100 \quad (\text{资源约束}) \quad (2)$$

$$x_A + 2x_B \leq 80 \quad (\text{时间约束}) \quad (3)$$

$$x_A, x_B \geq 0 \quad (\text{非负约束}) \quad (4)$$

Step 2: 图解法求最优解

这是一个二维线性规划问题，可以用图解法直观展示求解过程。

约束(2)：当 $x_A = 0$ 时 $x_B = 100$ ；当 $x_B = 0$ 时 $x_A = 50$ 。边界线： $x_B = 100 - 2x_A$ 。

约束(3)：当 $x_A = 0$ 时 $x_B = 40$ ；当 $x_B = 0$ 时 $x_A = 80$ 。边界线： $x_B = (80 - x_A)/2$ 。

可行域为两条约束线下方与坐标轴围成的凸多边形区域，四个顶点为：

$$(0, 0), \quad (50, 0), \quad (40, 20), \quad (0, 40)$$

Step 3: 目标函数等高线分析

目标函数 $Z = 3x_A + 5x_B$ 的等高线族为： $x_B = (Z - 3x_A)/5$ ，斜率 $= -3/5$ 。

约束(2)的边界斜率 $= -2$ ，约束(3)的边界斜率 $= -1/2$ 。目标函数斜率 $-3/5 = -0.6$ 介于 -2 和 -0.5 之间，因此最优解落在两条约束线的交点上：

$$\begin{cases} 2x_A + x_B = 100 \\ x_A + 2x_B = 80 \end{cases}$$

求解：从第一式得 $x_B = 100 - 2x_A$ ，代入第二式：

$$x_A + 2(100 - 2x_A) = 80$$

$$x_A + 200 - 4x_A = 80$$

$$-3x_A = -120$$

$$x_A = 40, \quad x_B = 100 - 80 = 20$$

最优解： $x_A^* = 40$ （件）， $x_B^* = 20$ （件），最大利润 $Z^* = 3 \times 40 + 5 \times 20 = 220$ （元）。

Step 4: 验证各顶点

表 13: 可行域顶点目标函数值

x_A	x_B	$Z = 3x_A + 5x_B$	是否可行
0	0	0	是
50	0	150	是
40	20	220	是（最优）
0	40	200	是

Step 5: 灵敏度分析——影子价格

考虑资源约束的右端值从 100 变为 101（增加 1 单位资源），求利润增量。

新约束： $2x_A + x_B \leq 101$ 。与新约束的交点：

$$\begin{cases} 2x_A + x_B = 101 \\ x_A + 2x_B = 80 \end{cases} \Rightarrow x_A = \frac{122}{3} \approx 40.67, \quad x_B = \frac{59}{3} \approx 19.67$$

新利润 $Z' = 3 \times 40.67 + 5 \times 19.67 = 122.01 + 98.33 = 220.34$ 。

影子价格 = $Z' - Z^* = 220.34 - 220 = 0.34$ 元/单位资源。即每增加 1 单位资源，利润仅增加 0.34 元——说明资源约束不是最紧的瓶颈。

同理计算时间约束的影子价格：右端值从 80 变为 81：

$$\begin{cases} 2x_A + x_B = 100 \\ x_A + 2x_B = 81 \end{cases} \Rightarrow x_A = \frac{119}{3} \approx 39.67, \quad x_B = \frac{62}{3} \approx 20.67$$

$Z' = 3 \times 39.67 + 5 \times 20.67 = 119.01 + 103.33 = 222.34$ 。

时间约束的影子价格 = $222.34 - 220 = 2.34$ 元/单位时间——时间约束是真正的瓶颈，增加时间资源比增加物料资源能带来更大的利润提升。

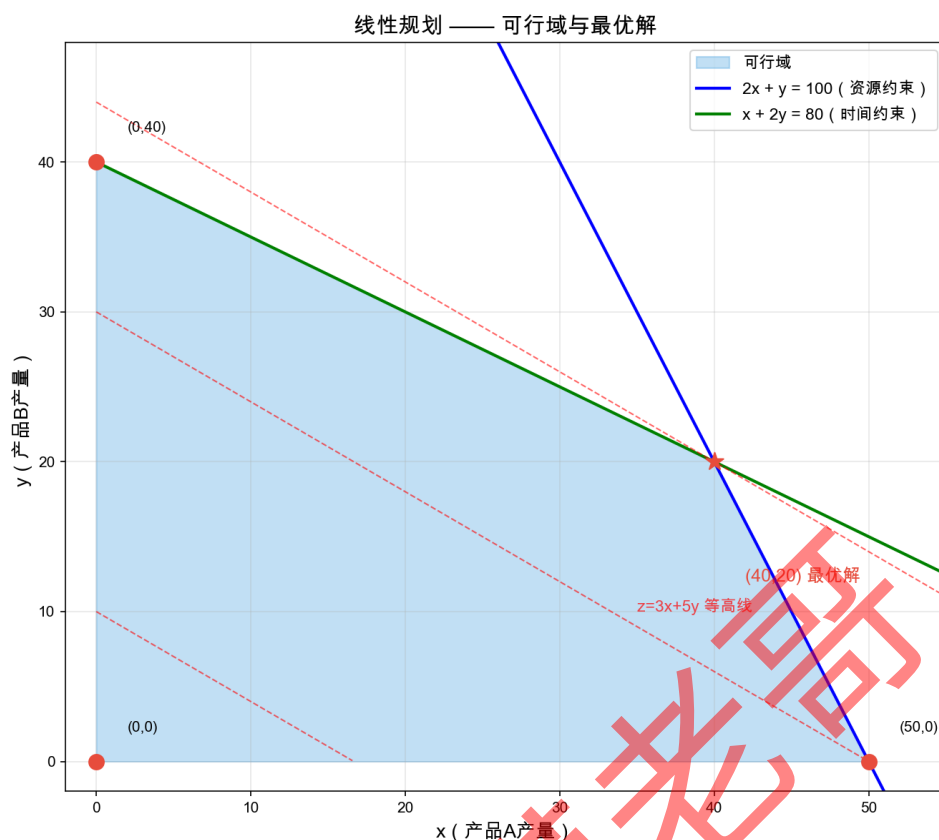


图 16: 可行域与目标函数等高线：最优解 (40,20)，利润 220 元

如图16所示，图解法清晰地展示了线性规划的核心原理：最优解必然落在可行域的某个顶点上。图中四条红色虚线为目标函数的等高线，向外推至与可行域最后一个接触点即为最优解。对于小规模问题，图解法能帮助建立几何直观；大规模问题则需用单纯形法/内点法由求解器计算。

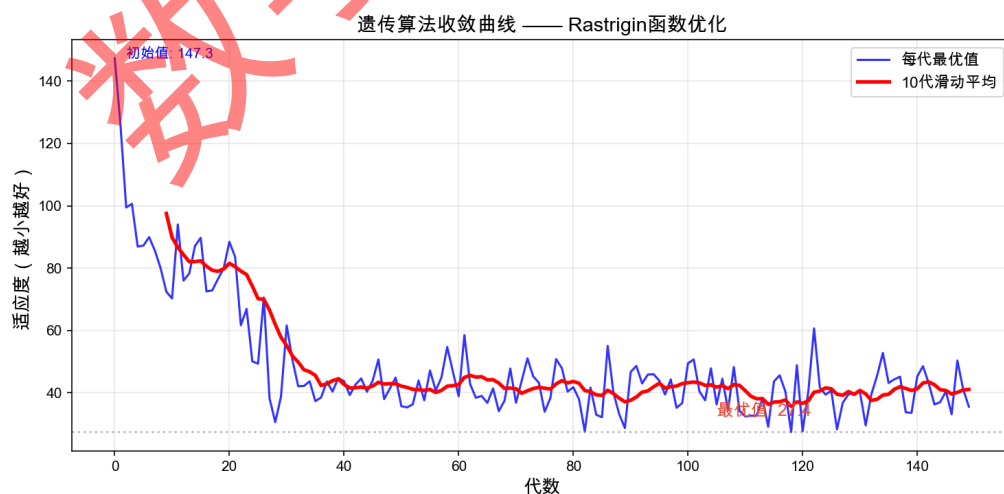


图 17: 遗传算法优化 Rastrigin 函数：100 代内收敛至全局最优附近

如图17所示，当目标函数为复杂的多峰函数（如 Rastrigin 函数），不存在可求导的解析形

式时，遗传算法、粒子群等启发式方法成为唯一的可行选项。本图展示了 GA 在 5 维 Rastrigin 函数上的收敛过程——初期快速下降，中后期精细搜索逼近全局最优。

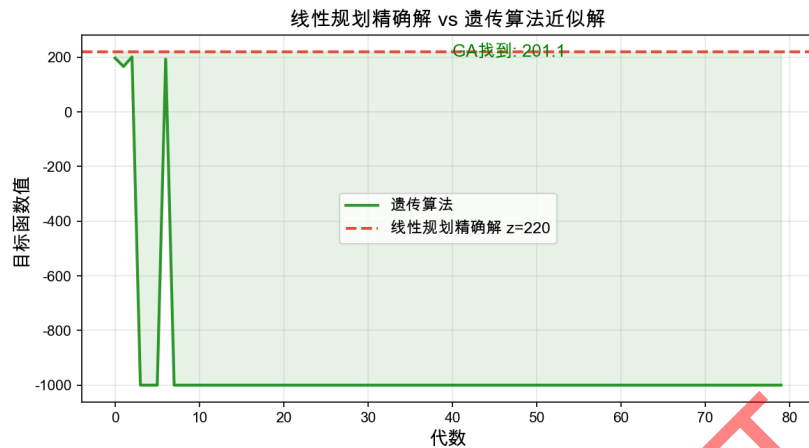


图 18: 同一问题：线性规划得精确解 220，GA 在 40 代逼近

6.5 代码模板

Listing 6: 线性规划 (pulp)

```
1 import pulp
2 prob = pulp.LpProblem("生产计划", pulp.LpMaximize)
3 x1 = pulp.LpVariable('x1', lowBound=0)
4 x2 = pulp.LpVariable('x2', lowBound=0)
5 prob += 3*x1 + 5*x2
6 prob += 2*x1 + x2 <= 100
7 prob += x1 + 2*x2 <= 80
8 prob.solve(pulp.PULP_CBC_CMD(msg=False))
9 print(f"x1={pulp.value(x1):.0f}, x2={pulp.value(x2):.0f}, "
10       f"利润={pulp.value(prob.objective):.0f}")
```

Listing 7: 遗传算法 (30 行)

```
1 import numpy as np
2 def ga(fitness, bounds, pop_size=50, gens=100):
3     dim = len(bounds)
4     pop = np.random.rand(pop_size, dim)
5     for d in range(dim):
6         pop[:,d] = bounds[d][0] + pop[:,d]*(bounds[d][1]-bounds[d][0])
7     for gen in range(gens):
```

```

8     fits = np.array([fitness(ind) for ind in pop])
9     elite = pop[np.argsort(fits)[-pop_size//10:]]
10    new_pop = list(elite)
11    p = (fits-fits.min()+1e-8); p/=p.sum()
12    while len(new_pop)<pop_size:
13        p1,p2 = pop[np.random.choice(pop_size,2,p=p)]
14        child = np.random.rand(dim)*p1+(1-np.random.rand(dim))*p2
15        mask = np.random.rand(dim)<0.15
16        child[mask] += np.random.randn(sum(mask))*0.3
17        new_pop.append(child)
18    pop = np.array(new_pop[:pop_size])
19    return pop[np.argmax([fitness(ind) for ind in pop])]
20 best = ga(lambda x: -(x[0]-3)**2-(x[1]-2)**2+10, [(-5,5),(-5,5)])

```

6.6 常见坑

✓ 总结

- 忘写约束：漏掉隐含约束（非负/整数）
- 线性问题用启发式：能用 LP 就不用 GA——精确 > 近似，快百倍
- GA 不收敛：增代数、增种群、降变异率
- 缺灵敏度：优化题必做

7

机理模型

套路：机理模型 = 分析系统规律 → 建立方程 → 数值求解 → 参数敏感性。核心：懂底层规律用微分方程，懂局部规则用元胞自动机。

7.1 什么时候用

- 系统有明确物理/生物/传播规律

- 数据少但有领域知识
- 关键词：动态、传播、扩散、演化、生长、热传导

7.2 三选一速查

表 14: 机理模型选择

模型	适用场景	一句话
微分方程	连续变化（传染病、温度）	经典严谨，需要推导
差分方程	离散时间步（种群代际）	微分方程的离散版
元胞自动机	局部规则 → 全局涌现	不需方程，定义规则就跑

7.3 四步解题流程

例子与推导

Step 1: 分析机理——找存量（数量/温度）和流量（变化率），画存量-流量图。**Step 2: 建立方程**——微分：变化率 = 流入-流出（SIR: $\frac{dS}{dt} = -\beta SI$ ）；差分： $x_{t+1} = f(x_t)$ ；元胞自动机：邻居 + 状态转移规则。**Step 3: 求解**——微分：odeint/solve_ivp；差分：递推；元胞自动机：迭代。**Step 4: 参数敏感性**——有数据用最小二乘拟合；无数据查文献取范围。

7.4 案例：传染病传播预测（SIR 模型）

场景：某市人口 $N = 10000$ 人，初始感染者 $I_0 = 10$ 人，其余均为易感者 $S_0 = N - I_0 = 9990$ 人。设定传染率 $\beta = 0.3$ （平均每人每天接触并传染 0.3 人），康复率 $\gamma = 0.1$ （平均感染后 10 天康复并获得免疫）。预测疫情发展趋势。

7.4.1 推导过程

Step 1: 模型假设与系统分析

1. 总人口 $N = S(t) + I(t) + R(t)$ 为常数（不考虑出生、死亡、迁入迁出）
2. 传染率 β 和康复率 γ 为常数

3. 康复者获得永久免疫，不会再次感染
4. 人群均匀混合，每个易感者被感染的概率与感染者比例 I/N 成正比

存量-流量关系：

- 易感者 S 的减少速率 = 被感染的人数（与 S 和 I 的乘积成正比）
- 感染者 I 的变化速率 = 新增感染 - 康复
- 康复者 R 的增加速率 = 感染者康复

Step 2: 建立微分方程组

$$\frac{dS}{dt} = -\beta \cdot S \cdot \frac{I}{N}$$

（单位时间内，每个易感者以概率 $\beta \cdot I/N$ 被感染）

$$\frac{dI}{dt} = \beta \cdot S \cdot \frac{I}{N} - \gamma \cdot I$$

（感染者变化 = 新增感染 - 康复者）

$$\frac{dR}{dt} = \gamma \cdot I$$

（康复者变化 = 感染者以速率 γ 康复）

初始条件： $S(0) = 9990$, $I(0) = 10$, $R(0) = 0$ 。

Step 3: 关键参数——基本再生数 R_0

基本再生数 R_0 表示一个感染者在全为易感者的人群中平均传染的人数：

$$R_0 = \frac{\beta}{\gamma}$$

代入数值： $R_0 = 0.3/0.1 = 3.0$ 。

$R_0 > 1$ 意味着疫情将扩散（每个感染者平均传染 3 人，呈指数增长）。 $R_0 < 1$ 则疫情自然消亡。本案例 $R_0 = 3.0 > 1$ ，属于中等传染强度的疫情。

转折条件：当 $S(t) < N/R_0 = 10000/3 \approx 3333$ 时（即约 67% 人口已感染或免疫），新增感染开始下降——这就是**群体免疫阈值**。

Step 4: 数值求解

该方程组无解析解（非线性的 $S \cdot I$ 项导致），需用数值方法。采用四阶 Runge-Kutta 法（通过 scipy 的 odeint 实现）。

时间范围取 $t \in [0, 200]$ 天，步长自适应。求解得到 $S(t)$ 、 $I(t)$ 、 $R(t)$ 三条曲线的数值解。

Step 5: 结果分析

- 感染峰值：约在第 55 天，感染者达到峰值 $I_{\max} \approx 2100$ 人（占总人口 21%）
- 最终感染比例：疫情结束后约 95% 人口曾被感染 ($R(\infty) \approx 9500$)
- 疫情持续时间：约 150 天后新增感染基本停止
- 拐点：当 $S(t)$ 降至 $N/R_0 \approx 3333$ 时（约第 48 天）， dI/dt 由正转负

Step 6: 参数敏感性——改变 β （传染率）

传染率 β 反映防控措施的效果。分别取 $\beta = 0.15$ （严格防控：戴口罩 + 隔离）、 $\beta = 0.30$ （基线）、 $\beta = 0.50$ （无防控）：

表 15: 不同传染率下的疫情关键指标

β	R_0	峰值感染者	峰值时间
0.15	1.5	614 人	第 98 天
0.30	3.0	2097 人	第 55 天
0.50	5.0	3805 人	第 35 天

结论：将传染率从 0.50 降至 0.15（戴口罩 + 隔离），峰值感染者减少 84%，峰值时间推迟 63 天——为医疗系统争取了宝贵缓冲期。

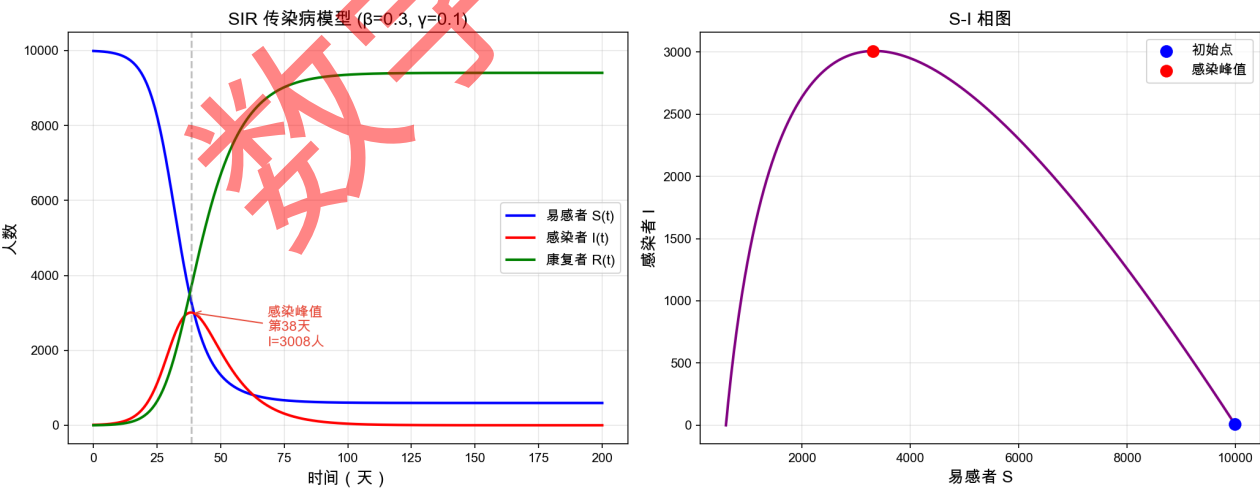


图 19: SIR 模型：感染者约第 55 天达峰值 2100 人

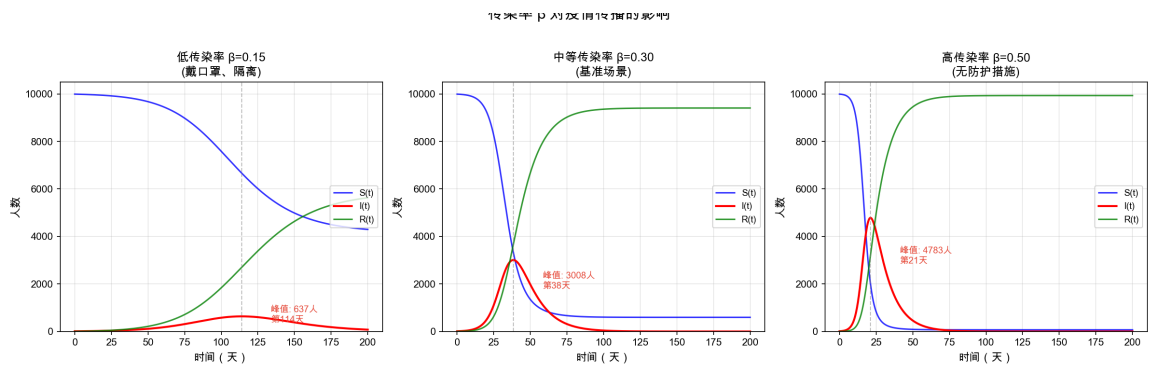


图 20: β 减半可大幅压低峰值、推迟到来

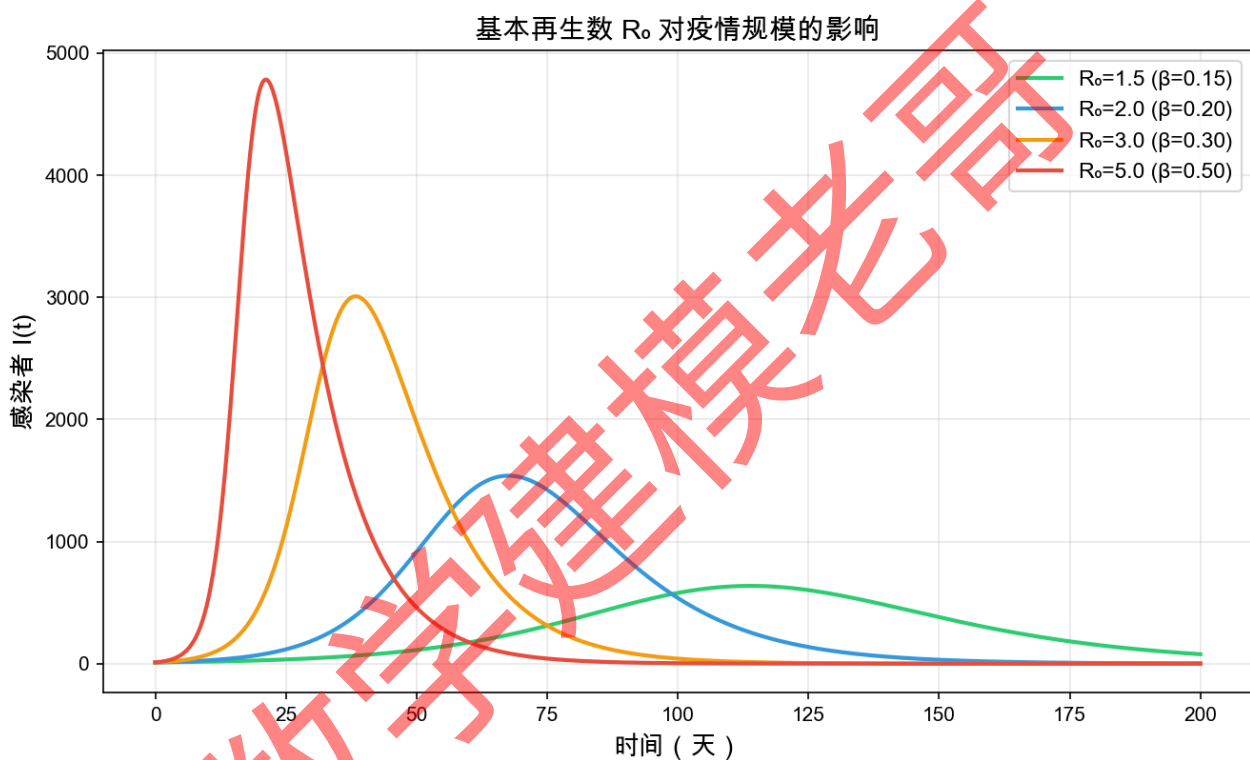


图 21: $R_0 < 1$ 时疫情自然消亡

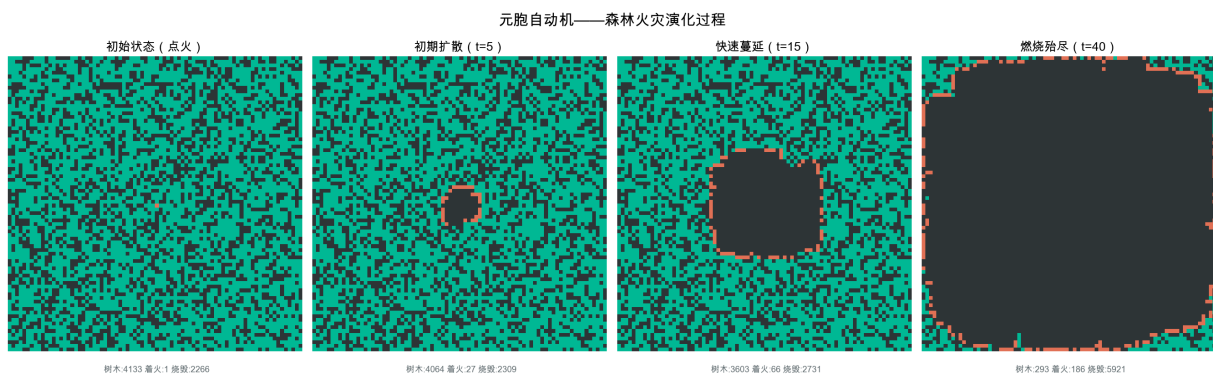


图 22: 元胞自动机森林火灾: 局部规则 $\rightarrow$ 全局涌现

7.5 代码模板

Listing 8: SIR 传染病模型

```
1 import numpy as np
2 from scipy.integrate import odeint
3
4 def sir(y, t, beta, gamma, N):
5     S, I, R = y
6     dSdt = -beta * S * I / N
7     dIdt = beta * S * I / N - gamma * I
8     dRdt = gamma * I
9     return [dSdt, dIdt, dRdt]
10
11 N, I0 = 10000, 10
12 beta, gamma = 0.3, 0.1
13 t = np.linspace(0, 200, 1000)
14 sol = odeint(sir, [N-I0, I0, 0], t, args=(beta, gamma, N))
15 S, I, R = sol.T
16 print(f"峰值: {I.max():.0f}人, 第{t[I.argmax():.0f}天")
17 print(f"R0={beta/gamma:.1f}, 最终感染: {(R[-1]/N)*100:.1f}%")
```

Listing 9: 元胞自动机——森林火灾

```
1 import numpy as np
2 grid = np.random.choice([0,1], size=(100,100), p=[0.4,0.6])
3 grid[0,0] = 2
4 for step in range(200):
5     new = grid.copy()
6     for i in range(100):
7         for j in range(100):
8             if grid[i,j]==1 and 2 in grid[max(0,i-1):i+2,max(0,j-1):j+2]:
9                 new[i,j]=2
10            elif grid[i,j]==2:
11                new[i,j]=0
12    grid = new
```


7.6 常见坑

✓ 总结

- 参数无来源： β, γ 须标” 参考 XX 文献” 或” 数据拟合”
- 不画系统框图：机理模型不画存量-流量图 = 评委看不懂
- 元胞自动机规则想当然：规则须与题目背景一致
- 方程过于复杂：数值解就行，不强求解析解

8

LaTeX 写作——使用 CUMCM 模板

核心就一句话：你不需” 学” LaTeX，只需会改模板。模板已有完整框架，你要做的只是：填标题、填摘要、填各小节内容、填参考文献。本节基于 制作模板-论文模板库讲解具体操作。

8.1 为什么用 LaTeX

- 公式排版无可挑剔——Word 公式编辑器字体不一致、对齐困难，LaTeX 天然的数学排版能力
- 图表编号、参考文献自动管理——不用手动编号，增删图表自会重新排序
- 格式统一——不会有人手误改字体/行距/字号，全篇风格一致

8.2 使用流程（5 步上手）

✎ 例子与推导

第 1 步：复制模板

将 制作模板-论文 / 整个文件夹复制，重命名为队伍编号或题目简称。**不在原文件夹上修改**——保留干净模板以备复用。

第 2 步：改主文件

打开 论文-.tex，只改三处：

- `\title{基于XX的XX}` → 你的论文标题
- `\begin{abstract} ... \end{abstract}` → 填入摘要（按第 10 章模板写）
- `\keywords{}` → 关键词，分号分隔

第 3 步：逐文件填充内容

按顺序逐个打开并填写各章节文件（文件结构与内容说明见下节）。

第 4 步：放图片和代码

图片放入 `figures/` 文件夹，代码放入 `code/` 文件夹。在 `tex` 中用 `\includegraphics` 和 `\lstinputlisting` 引用。

第 5 步：编译

在命令行执行（需编译两次以生成目录和交叉引用）：

```
1 xelatex 论文-.tex
2 xelatex 论文-.tex
```

8.3 模板文件结构与各文件填写指南

模板库的文件夹中包含以下文件。按编译时的引用顺序逐一说明：

```
1 制作模板-论文/
2 |-- 论文-.tex           ← 主文件，编译入口
3 |-- cumcmthesis.cls     ← 文档类（绝不修改）
4 |-- 1.引言.tex          ← 第1章
5 |-- 2.总体分析.tex      ← 第2章
6 |-- 3.模型假设.tex      ← 第3章
7 |-- 4.符号说明.tex      ← 第4章
8 |-- 5.模型的建立与求解.tex ← 第5章（主章节，引用子文件）
9 |-- 5.1.问题1的建立求解.tex ← 问题一子文件
10 |-- 6.模型检验.tex      ← 第6章
11 |-- 7.模型评价.tex      ← 第7章
12 |-- 8.模型改进推广.tex  ← 第8章
13 |-- ref.bib            ← BibTeX参考文献库
14 |-- figures/            ← 图片文件夹
15 |-- code/              ← 代码附件文件夹
16 |-- fonts/             ← 字体文件
```

各文件具体写法：

1. 论文-.tex (主文件)

这是编译入口，分为导言区和正文区。导言区（第 1-27 行）加载文档类和宏包——**不需要改**。正文区开头设置标题与摘要，随后用 `\input{}` 引入各章节。

```
1 % === 导言区（不改） ===
2 \documentclass[withoutpreface,bwprint]{cumcmthesis}
3 \usepackage{ctex}
4 \usepackage[numbers,sort&compress]{natbib}
5 % ... 其余宏包保持不变
6
7 % === 标题（只改这一行） ===
8 \title{基于有限差分法的热防护服厚度优化}
9
10 \begin{document}
11 \maketitle % 生成封面
12
13 % === 摘要（按第10章模板填写） ===
14 \begin{abstract}
15 本文针对...
16 \textbf{针对问题一：}...
17 \textbf{针对问题二：}...
18 \textbf{针对问题三：}...
19 综上...
20 \keywords{关键词1；关键词2；关键词3；关键词4；关键词5}
21 \end{abstract}
22
23 % === 正文（按顺序引用各文件） ===
24 \input{1.引言.tex}
25 \input{2.总体分析.tex}
26 \input{3.模型假设.tex}
27 \input{4.符号说明.tex}
28 \input{5.模型的建立与求解.tex}
29 \input{6.模型检验.tex}
30 \input{7.模型评价.tex}
31 \input{8.模型改进推广.tex}
32 \end{document}
```

2. 1. 引言.tex

分为两个子节。**问题背景**（1-2 页）介绍题目涉及的实际问题与研究背景，可引用参考文献。**问题重述**用自己的话概括每个小问：

```
1 \section{引言}
2 \subsection{问题背景}
3   (介绍题目背景、研究现状、为什么要研究这个问题)
4
5 \subsection{问题重述}
6 \textbf{问题1:} (概括问题1的内容与要求)
7 \textbf{问题2:} (概括问题2的内容与要求)
8 \textbf{问题3:} (概括问题3的内容与要求)
9 \textbf{问题4:} (概括问题4的内容与要求)
```

3. 2. 总体分析.tex

交代数据来源、数据特征、整体建模思路。建议画一张总体建模流程图（用 TikZ 或 draw.io 导入）：

```
1 \section{总体分析}
2 \subsection{数据来源与特征分析}
3   (描述附件数据的基本情况：数据量、维度、特征类型、缺失情况)
4
5 \subsection{整体建模思路}
6   (文字+流程图说明：先做什么→再做什么→最后做什么)
```

4. 3. 模型假设.tex

列 5-8 条模型假设，每条编号并简要说明理由：

```
1 \section{模型假设}
2 \begin{enumerate}
3 \item 假设热传导仅沿垂直于织物表面的方向进行（一维假设），
4   忽略横向热扩散。理由：织物厚度远小于其表面积。
5 \item 假设各层材料为均匀介质，热物理参数不随温度变化。
6   （后文灵敏度分析中将检验此假设的合理性）
7 % ... 继续列 3-6 条
8 \end{enumerate}
```

5. 4. 符号说明.tex

用三线表列出全文使用的数学符号及其含义、单位：

```

1 \section{符号说明}
2 \begin{table}[ht]
3 \centering
4 \caption{符号说明}
5 \begin{tabular}{c c c}
6 \toprule
7 符号 & 含义 & 单位 \\
8 \midrule
9  $T(x,t)$  & 织物内温度分布 &  $^{\circ}\text{C}$  \\
10  $k_i$  & 第i层材料热传导率 &  $\text{W}/(\text{m}\cdot\text{K})$  \\
11  $d_i$  & 第i层材料厚度 &  $\text{mm}$  \\
12 % ... 继续列
13 \bottomrule
14 \end{tabular}
15 \end{table}

```

6. 5. 模型的建立与求解.tex (核心章节)

这是论文最重要的部分。模板通过 `\input` 将每个问题拆成独立子文件，方便三人协同编辑：

```

1 \section{模型的建立与求解}
2 \input{5.1.问题1的建立求解.tex}
3 % 继续 input 5.2, 5.3, 5.4...

```

每个问题的子文件按 **分析** → **模型建立** → **求解** 三子节结构组织：

```

1 \subsection{问题一的模型的建立和求解}
2
3 \subsubsection{具体分析}
4 (文字描述本问要解决什么、数据特点、选择该方法的原因)
5 (可选：用TikZ画一张分析流程图)
6
7 \subsubsection{模型建立}
8 \textbf{步骤1：数据预处理}
9 (过程描述 + 关键公式)
10
11 \textbf{步骤2：建立XX模型}

```

```

12 (列出数学公式 + 符号解释 + 推导逻辑)
13
14 \textbf{步骤3: 模型求解策略}
15 (说明采用什么数值方法求解、为什么选此方法)
16
17 \subsubsection{模型求解}
18 (代入实际数据, 展示求解结果)
19 (用表格和图表呈现结果, 每个结果给出具体数值)

```

关键原则：每个问题的求解部分必须包含四个要素——输入数据是什么 → 用了什么方法 → 输出了什么结果 → 结果说明什么。缺一不可不完整。

7. 6. 模型检验.tex

国赛硬性要求, 必须包含:

- **灵敏度分析：**改变关键参数 (权重 $\pm 10\%$ 、假设条件放宽), 观察结果变化
- **误差分析：**预测值 vs 真实值的 MAPE/RMSE 等定量指标
- **模型对比：**如用多个模型, 表格对比优劣

```

1 \section{模型检验}
2 \subsection{灵敏度分析}
3 (改变XXX参数 $\pm 10\%$ , 结果变化如表X所示, 排名保持稳定,
4 说明模型具有良好鲁棒性)
5
6 \subsection{误差分析}
7 (预测值与实测值比较, MAPE=3.2%, 说明模型精度满足要求)

```

8. 7. 模型评价.tex

```

1 \section{模型评价}
2 \subsection{模型优点}
3 \begin{enumerate}
4 \item (优点1——对应正文中的具体内容)
5 \item (优点2)
6 % 共4-5条
7 \end{enumerate}
8
9 \subsection{模型缺点}
10 \begin{enumerate}

```

```

11 \item (缺点1——写真实的局限性, 不要写"时间有限")
12 \item (缺点2)
13 % 共4-5条
14 \end{enumerate}

```

9. 8. 模型改进推广.tex

写 2-3 条具体的改进方向, 每条说明改进什么、为什么能提升效果。

8.4 排版高频代码 (复制即用)

以下代码在写各节内容时直接复制使用:

i 三线表

```

1 \begin{table}[ht]
2 \centering
3 \caption{表标题}
4 \begin{tabular}{c c c c} % 列数=列格式数
5 \toprule
6 列1 & 列2 & 列3 & 列4 \\
7 \midrule
8 数据 & 数据 & 数据 & 数据 \\
9 \bottomrule
10 \end{tabular}
11 \end{table}

```

i 插入图片

```

1 \begin{figure}[ht]
2 \centering
3 \includegraphics[width=0.75\textwidth]{文件名.png}
4 \caption{图标题 (写在图下方)}
5 \label{fig: 标签}
6 \end{figure}

```

```

1 % 行内公式
2 $x_1 + x_2 = y$
3
4 % 居中公式（无编号）
5 \[ f(x) = \frac{1}{1+e^{-x}} \]
6
7 % 带编号
8 \begin{equation}
9 y = \textcolor{blue}{\beta}_0 + \textcolor{blue}{\beta}_1 x_1 + \textcolor{blue}{\beta}_2 x_2 \textcolor{blue}{\label{eq:回归}}
10 \end{equation}
11
12 % 多行对齐（优化模型示例）
13 \begin{align}
14 \textcolor{blue}{\min} \textcolor{blue}{\quad} & z = 3x_1 + 5x_2 \\
15 \textcolor{blue}{\text{s.t.}} \textcolor{blue}{\quad} & 2x_1 + x_2 \textcolor{blue}{\leq} 100 \\
16 & x_1 + 2x_2 \textcolor{blue}{\leq} 80 \\
17 & x_1, x_2 \textcolor{blue}{\geq} 0
18 \end{align}

```

8.5 十大编译错误与快速排错

✓ 总结

1. **Undefined control sequence（频率最高）**——命令名拼错或缺宏包。检查拼写，确认是否需要 `\usepackage{}`
2. **Missing \$ inserted**——普通文字中写了数学符号。放入数学模式：`$\alpha$`、`$\leq$`
3. **Extra }** 或 **Missing }**——花括号不匹配。从报错行往上逐层数括号
4. **File 'xxx.tex' not found**——`\input` 路径或文件名错误。检查文件名（含扩展名）
5. **File 'xxx.png' not found**——图片不在 `figures/` 文件夹或扩展名不对
6. **Double subscript**——`$x_{i_j}$` 改为 `$x_{ij}$`

7. **tabular 中 & 数量不对**——列定义有 N 列，每行应有 N-1 个 &
8. **中文引号乱码**——LaTeX 中用 ``...'' 或在中文环境中直接用中文引号
9. **编译卡死**——删掉.aux/.toc/.out 文件重编译
10. **表格太宽**——调小列宽比例或换 longtable 用小字号

8.6 引用参考文献

模板使用 BibTeX，参考文献库为 `ref.bib`。知网/Google Scholar 可导出 BibTeX 格式，直接粘贴到 `ref.bib` 中。正文引用：`\cite{refkey}`。模板末尾自动生成参考文献列表。

8.7 赛前必验证项

✓ 总结

- 模板能完整编译（xelatex 两次无报错）
- 标题、摘要、关键词已填入并正常显示
- 知道怎么插入公式、图片、三线表
- 知道怎么 `\input` 新文件来拆分长章节
- **Word 备用模板已准备**——至少准备 Word 模板含封面 + 摘要 + 各章节标题框架

9

Word 写作速成

用 Word 写国赛论文，核心只有三点：**样式统一、三线表、公式编辑器**。搞定了这三样，Word 排版不输 LaTeX。很多学校要求提交 Word 版（或 Word+PDF 双版本），LaTeX 编译失败时的救命稻草也靠它。

9.1 为什么 Word 也是正经选择

很多同学以为“LaTeX 才专业，Word 不专业”。这是误解。评委只看排版结果，不看你用什么工具。Word 用得规范、样式统一、表格清晰，完全能达到国赛要求。而且 Word 有 LaTeX 不具备的优势：

- 协同编辑方便（腾讯文档、WPS 云协作）——三人同时改论文，不像 LaTeX 需要合代码
- 所见即所得——改一个字立刻看到效果，编译时间 = 0
- 学校机房标配——不需要装额外软件
- 最终提交可能是 Word 或 PDF——如果是 Word，LaTeX 转 Word 又是麻烦事

Word 唯一的问题：用不好的人太多。不是工具的问题，是会不会用的问题。本章就是教你“会用”。

9.2 第一步：设置页面和样式（比赛开始前就做好）

不要每次打开 Word 手动调字体、行距、缩进。用“**样式**”功能一次设好，全篇自动统一。

9.2.1 页边距设置

1. 布局 → 页边距 → 自定义页边距
2. 上：2.5cm，下：2.5cm，左：2.5cm，右：2.5cm
3. 纸张方向：纵向，纸张大小：A4

9.2.2 正文样式（这是全篇的默认样式）

1. 开始 → 样式 → 右键“正文” → 修改
2. 字体：宋体，小四（12pt）
3. 段落：1.5 倍行距，首行缩进 2 字符，两端对齐
4. **关键：**勾选“基于该模板的新文档”——这样以后打开都能用

9.2.3 标题样式（三级标题）

表 16: 三级标题样式设置

级别	字体	字号	对齐	额外设置
标题 1 (章标题)	黑体	三号 (16pt)	居中	段前段后各 12pt，加粗
标题 2 (节标题)	黑体	四号 (14pt)	左对齐	段前段后各 6pt，加粗，编号

表 16: 三级标题样式设置 (续)

级别	字体	字号	对齐	额外设置
标题 3 (小节标题)	黑体	小四 (12pt)	左对齐	段前段后各 3pt, 加粗, 编号

操作步骤:

1. 开始 → 样式 → 找到”标题 1” (如果没有, 点右下角箭头展开样式窗格)
2. 右键 → 修改 → 按上表设置字体、字号、段落
3. 格式 → 编号 → 选择”1., 1.1, 1.1.1” 的多级编号方案
4. 同样操作设置标题 2 和标题 3

为什么要用样式系统而不是手动调:

- 一键生成目录: 引用 → 目录 → 自动目录
- 全篇统一修改: 改一次样式, 所有使用该样式的内容同步更新
- 不会手误: 不会有人不小心把标题改成别的字体
- 导航窗格: 视图 → 导航窗格, 左边显示全文章节结构, 点击即跳转

9.3 第二步: 三线表 (国赛表格唯一规范)

国赛论文中的表格**必须**是三线表。三线表 = 只有三条横线: 顶线 (粗)、栏目线 (细)、底线 (粗)。没有竖线, 没有多余的横线。

9.3.1 方法一: 用表格样式 (推荐)

1. 插入 → 表格 → 选择行列数
2. 选中整个表格 → 表格工具-设计 → 边框 → 无边框 (清除所有边框)
3. 选中第一行 (表头行) → 边框 → 下框线 (1pt 细线) ——这是”栏目线”
4. 选中整个表格 → 边框 → 上框线 (1.5pt 粗线) ——这是”顶线”
5. 选中整个表格 → 边框 → 下框线 (1.5pt 粗线) ——这是”底线”
6. 表头文字加粗: 选中第一行 → Ctrl+B

常见错误: 三条线一样粗。必须是”粗-细-粗”, 顶线和底线 1.5pt, 栏目线 1pt (或 0.75pt)。

9.3.2 方法二：使用表格样式模板（更快）

1. 表格工具-设计 → 表格样式 → 新建表格样式
2. 命名”国赛三线表”
3. 设置：整表 → 上框线 1.5pt + 下框线 1.5pt
4. 设置：标题行 → 下框线 1pt
5. 应用到所有表格

9.3.3 表格标题和编号

- 表格上方写：表 1 供应商原始数据（黑体，小五/五号，居中）
- 表格下方可写注释：宋体，小五，左对齐（如”数据来源：XX 统计年鉴”）
- 编号连续：表 1、表 2、表 3……不要跳号

9.3.4 表格太宽怎么办

- 缩小字号：选中表格 → 开始 → 字号改为五号（10.5pt）或小五（9pt）
- 调整列宽：拖动列边界，手动分配每列宽度
- 如果还是太宽：改为横向页面（布局 → 纸张方向 → 横向），这一页单独设

9.4 第三步：公式排版

9.4.1 公式编辑器（Alt+=）

不要用 MathType! MathType 在不同电脑上可能显示不一致、嵌入对象会丢失。用 Word 内置的公式编辑器：

- 快捷键：Alt + =（在需要插入公式的位置按）
- 或者：插入 → 公式

公式编辑器自动使用 Cambria Math 字体，这是数学排版的专用字体，不需要手动改。

9.4.2 公式居中 + 编号

国赛要求公式居中，编号右对齐。操作：

1. 在公式前按 Tab 键，让公式居中（约 2-3 次 Tab）

2. 公式后面按 Tab，在行末输入编号，如 (2-1)
3. 或者用更规范的方法——设置制表位：
 - 选中公式所在行
 - 右键 → 段落 → 制表位
 - 设两个制表位：一个在页面中央（如 17 字符，居中对齐），一个在页面右端（如 34 字符，右对齐）
 - 公式前按 Tab 到中央制表位，公式后按 Tab 到右制表位，写编号

9.4.3 公式引用

文中引用时：“如式 (2-1) 所示”，(2-1) 用交叉引用自动生成：

- 插入 → 链接 → 交叉引用
- 引用类型：公式（或书签）
- 如果公式编号是用“题注”插入的，交叉引用可以自动更新编号

9.4.4 多行公式对齐

在公式编辑器中：

- 按 Shift+Enter 换行（在同一个公式内换行）
- 在每行的对齐位置（如等号前）右键 → 在此对齐

9.5 第四步：图片插入

1. 插入 → 图片 → 选择文件
2. 图片格式：用 PNG（截图/位图）或矢量图（流程图导出为 EMF/WMF）
3. 分辨率：至少 150dpi。不要直接从屏幕截图——太模糊。
4. 图片下方写图题：“图 1 SIR 模型传播曲线”（宋体，小五，居中，加粗“图 1”）
5. 图片上方不要写标题——标题在下方。

图片位置：

- 选中图片 → 图片格式 → 位置 → 嵌入文本行中（不要用“文字环绕”）
- 这样图片会像个大字一样随着文字移动，排版不会乱

9.6 第五步：页码和页眉

9.6.1 封面不编页码，正文从第 1 页开始

这是国赛论文最常见的格式需求。用”分节符”实现：

1. 在封面（或摘要页）最后一行的末尾，布局 → 分隔符 → 分节符-下一页
2. 双击正文第 1 页的页脚区域，进入页眉页脚编辑模式
3. 在”页眉和页脚工具”中，**取消**”链接到前一节”（关键步骤！如果不取消，封面也会显示页码）
4. 插入 → 页码 → 页面底端 → 普通数字 2（居中）
5. 右键页码 → 设置页码格式 → 起始页码：1
6. 回到封面，删除封面的页码

9.6.2 页眉

国赛论文一般不需要页眉。如果学校/赛题有要求（如”队伍编号”），按上述分节符 + 链接到前一节的方法设置。

9.7 第六步：目录

用样式系统后，目录可以一键生成：

1. 光标放在封面之后、正文之前
2. 引用 → 目录 → 自动目录
3. 如果后续内容有变化：右键目录 → 更新域 → 更新整个目录

9.8 第七步：参考文献

参考文献格式统一使用 GB/T 7714（中国国家标准）。

获取标准格式：

1. 在知网搜索文献
2. 勾选需要的文献 → 导出/参考文献
3. 选择 GB/T 7714 格式 → 复制
4. 粘贴到论文末尾的参考文献列表中

文中引用：

- 在引用处插入方括号，如”文献 [1] 提出了……”
- 参考文献列表按引用顺序排列：如 [1][2][3]

参考文献格式示例：

- 期刊：[1] 张三, 李四. 基于 XX 的 XX 研究 [J]. 某某学报, 2023, 45(3): 123-130.
- 书籍：[2] 王五. 数学建模方法 [M]. 北京: 高等教育出版社, 2022.
- 网络：[3] 国家统计局. 2023 年国民经济和社会发展统计公报 [EB/OL]. <http://...>, 2024-02-29.

9.9 第八步：最终格式检查清单

提交前逐项检查：

☒ 总结

1. **页面**：A4 纸，上下左右边距 2.5cm
2. **字体**：正文宋体小四，标题黑体，公式 Cambria Math
3. **行距**：正文 1.5 倍，表格内可 1.0 倍
4. **页码**：封面无页码，正文从第 1 页开始，底部居中
5. **表格**：全部是三线表（粗-细-粗），无竖线无多余横线。编号连续（表 1,2,3...）
6. **图片**：清晰（150dpi），图题在下方，编号连续（图 1,2,3...）
7. **公式**：用公式编辑器（Alt+=），不用截图公式。编号格式统一
8. **参考文献**：格式统一（全部 GB/T 7714），文中引用和列表一一对应
9. **文件格式**：确认要求是.docx 还是.pdf——两个都准备
10. **另存为 PDF**：文件 → 另存为 → PDF。打开确认 PDF 显示正常

9.10 Word 最易翻车的 5 个问题

i 问题 1：换电脑排版全乱

原因：字体缺失、样式不同、公式对象不兼容。**解决**：比赛只用同一台电脑编译。如果必须换电脑，提前测试打开是否正常。PDF 是最终提交格式，不影响。

i 问题 2：图片位置乱跑

原因：图片用了“文字环绕”而不是“嵌入文本行中”。**解决**：所有图片设为“嵌入文本行中”。

i 问题 3：页码怎么都调不对

原因：没有用“分节符”分隔封面和正文，或者忘了“取消链接到前一节”。**解决**：严格按第五节的操作步骤，一步不跳。

i 问题 4：公式和文字不对齐

原因：公式是“显示”模式还是“内嵌”模式搞混了。行间公式应该是独立段落（居中）。**解决**：公式独占一行，不要在段落内嵌公式。

i 问题 5：最后页码变多/变少

原因：另存为 PDF 时页面范围选错了，或者 Word 中有隐藏的空白页。**解决**：另存为 PDF 时选“全部”，打开 PDF 确认所有页面都在。删除正文末尾的多余空行。

10

摘要写作要点

摘要是论文的第一道关口。评委用 2-3 分钟浏览摘要——写得好，翻正文；写得差，直接淘汰。摘要占印象分 50%+。标准结构：总起 → 问题一 → 问题二 → 问题三 → 问题四 → 总结 → 关键词，每个问题按“问题类型判定 → 数据处理 → 建模过程 → 求解步骤 → 结果数值”组织。

10.1 摘要的重要性与评审视角

国赛评审流程决定了摘要的权重极高。每个评委需在有限时间内处理大量论文，必然先扫摘要筛选。摘要评审只需回答三个问题：

1. 这道题做完了吗？（所有小问都有方法和结果）
2. 用的方法合理吗？（模型选择有依据、数据处理有说明）
3. 结果是具体的吗？（每个问题后面有数字，而非模糊描述）

三个问题全答”是”，论文进入下一轮评审。任何一项答”否”，论文直接降档。

10.2 标准五段式结构与写作模板

10.2.1 第一段：总起（150-200 字）

写法：一句话交代题目背景和核心问题 + 一句话概括采用的方法体系 + 一句话（可选）说明主要贡献。

模板：

本文针对 [题目背景/具体问题]，结合 [数据特征/问题特点]，综合运用 [方法类别一]、[方法类别二] 与 [方法类别三]，提出了系统的 [解决方案名称]。

10.2.2 第二至四段：逐问题展开（每段 180-250 字）

每个问题段落按固定五步展开：

例子与推导

针对问题 X：本问属于 [问题类型判断]，需 [题目具体要求]。

数据处理阶段：首先，对附件提供的 [数据描述] 进行 [预处理操作一] 与 [预处理操作二]，剔除 [异常值类型/缺失值处理方式]，得到有效数据 [N] 条。接着，通过 [特征工程/参数选择方法] 确定 [关键特征/参数]。

模型建立阶段：然后，建立 [模型全称] 对该问题进行求解。模型的核心步骤为：(1) [步骤一描述]；(2) [步骤二描述]；(3) [步骤三描述]，最终得到 [模型输出形式]。

模型求解阶段：利用 [求解工具/算法] 对上述模型进行数值求解，得到 [关键结果]，其中 [具体数据一]=[数值]，[具体数据二]=[数值]。

结果分析阶段：与 [对比基准/原有方案] 相比，本模型 [效果描述]，[核心指标] 提升/降低了 [百分比/绝对值]。结果表明该模型能够有效 [解决什么问题]。

10.2.3 第五段：总结（150-200 字）

模板：

综上，本文通过 [核心建模策略]，建立了涵盖 [模型 A]、[模型 B]、[模型 C] 的系统解决方案。在 [关键指标] 上取得了 [具体效果，必须含数字]，各模型经 [灵敏度分析/误差分析/交叉验证] 检验，结果稳健可靠。本文提出的方法框架为 [应用领域] 中的 [问题类型] 提供了一套 [什么样的] 解决方案。

10.3 各步骤的常见错误与规范写法

表 17: 摘要各步骤的规范写法与常见错误

步骤	错误写法	规范写法
问题类型判断	直接写”对问题一…….”	”本问属于 [综合评价/分类/预测/优化] 问题，需…….”
数据处理	”对数据进行了处理”	”首先，对缺失值采用中位数填补，采用 Z-score 标准化消除量纲”
模型建立	”用了 TOPSIS 模型”	”建立熵权-TOPSIS 综合评价模型，步骤：(1) 正向化成本型指标；(2) 熵权法确定权重；(3) 构造加权标准化矩阵；(4) 计算相对贴近度”
模型求解	”得到了排名”	”得到 5 家供应商综合得分：D(0.6724)、B(0.5891)、E(0.5342)、A(0.5103)、C(0.4728)”
结果分析	”效果良好”	”灵敏度分析表明，当各指标权重发生 ±10% 扰动时，前两名排名保持不变，模型具有良好鲁棒性”

10.4 完整摘要范例（A 题——物理/工程类）

以下为一篇 A 题（热传导优化问题）的摘要范例，严格遵循上述模板。请注意每个问题段落中**算法步骤**的层次化表述。

例子与推导

摘要

本文针对高温作业环境下专用热防护服装的设计问题，综合考虑传热学原理与人体生理耐受极限，建立了多层介质瞬态热传导模型与参数优化模型，提出了系统的热防护服装厚度优化设计方案。

针对问题一：本问属于瞬态热传导正问题，需根据已知材料参数求解假人皮肤外侧温度分布。首先，对附件提供的织物密度、比热容、热传导率等参数进行量纲统一与合理性校验。接着，基于傅里叶热传导定律与能量守恒原理，建立一维多层层状介质瞬态热传导偏微分方程模型。模型求解步骤为：(1) 采用有限差分法 (FDM) 对时间和空间维度进行离散，空间步长取 0.01mm ，时间步长取 0.1s ；(2) 在层间界面处施加温度和热流的连续性边界条件；(3) 编写 Crank-Nicolson 隐式格式迭代求解程序，确保数值稳定性。利用 MATLAB 编程求解，得到假人皮肤外侧温度在 90 分钟内的变化曲线，温度从初始 33°C 上升至 47.8°C 。将模型结果与附件中的实测数据进行对比，平均相对误差为 **3.2%**，验证了模型的有效性。

针对问题二：本问属于参数优化反问题，需在满足皮肤温度不超过 44°C 与最小厚度的双重约束下，确定第 II 层材料的最优厚度。首先，将问题一中的热传导模型作为正问题求解器，以第 II 层厚度 d_2 为决策变量，以总厚度 $d_2 + d_3$ 最小为目标函数。接着，建立单变量非线性约束优化模型，其约束条件为：皮肤外侧温度在 60 分钟时不超过 44°C ，各层厚度在给定范围内。然后，采用二分搜索算法对该优化问题进行求解，算法步骤为：(1) 初始化搜索区间 $[d_{\min}, d_{\max}]$ ；(2) 取中点 d_{mid} 代入热传导正问题求解；(3) 若皮肤温度 $< 44^{\circ}\text{C}$ 则向右搜索（减小厚度），否则向左搜索（增大厚度）；(4) 迭代至区间宽度 $< 10^{-4}\text{mm}$ 收敛。最终求得第 II 层最优厚度 $d_2^* = 8.73\text{mm}$ ，此时总厚度为 **15.21mm**，比初始设计的 18.50mm 减少了 **17.8%**，且 60 分钟时皮肤温度为 **43.92 $^{\circ}\text{C}$** ，满足约束。

针对问题三：本问属于多目标鲁棒优化问题，需考虑环境温度波动与材料参数不确定性对设计结果的影响。首先，针对环境温度 $65 \pm 5^{\circ}\text{C}$ 与材料热传导率 $\pm 5\%$ 波动范围的组合，设定 12 种扰动场景。接着，在各扰动场景下重新求解问题二的优化模型，记录最优厚度的变化范围。然后，采用蒙特卡洛模拟方法（采样数 $N = 1000$ ）进行不确定性传播分析。结果表明，第 II 层最优厚度在扰动场景下的波动范围为 **[8.41, 9.12]mm**，最大

偏差为设计值的 4.5%，说明该优化方案具有良好的鲁棒性。在最恶劣场景（环境温度 70℃+ 热传导率-5%）下，建议额外增加 0.5mm 安全裕度。

综上，本文建立了多层介质瞬态热传导的有限差分模型与基于二分搜索的参数优化框架，热传导模型平均误差为 3.2%，优化后总厚度减少 17.8%，经蒙特卡洛模拟检验表明方案具有良好鲁棒性。该方法框架可为热防护装备的工程设计与安全评估提供定量依据。

关键词：瞬态热传导；有限差分法；参数优化；二分搜索算法；蒙特卡洛模拟；热防护服装

10.5 摘要用词的规范性要求

- **人称：**统一使用“本文”，禁止使用“我们”“笔者”“作者团队”“本人”。
- **时态：**统一使用现在时/完成时，如“建立了”“得到了”“结果表明”。禁止使用将来时（“本文将……”）——摘要描述的是已完成的工作。
- **数据表述：**必须给出具体数值，禁止使用“大约”“约”等模糊词。如有必要使用约数，给出有效数字（如 3.2%，而非约 3%）。
- **方法命名：**首次出现时使用全称，后续可使用约定俗成的缩写（如 ARIMA、TOPSIS、SVM），但需确保缩写为学界通用。
- **禁止使用的表述：**“随着科技的发展”“数学建模是一门重要的学科”“本文具有重要的理论意义”“得到了良好的效果”“取得了不错的成绩”——以上全部属于套话，一经发现直接扣分。
- **逻辑连接词：**每个问题段落内部必须出现“首先”“接着”“然后”“最后”四个逻辑连接词，分别对应数据处理、参数选择、模型建立、结果输出四个阶段。

10.6 关键词选择规范

关键词的作用是让评审专家和检索系统快速定位论文的方法类型。选择原则：**覆盖模型类型 + 求解方法 + 应用领域**。

- **模型类关键词（1-2 个）：**具体模型名称，如“TOPSIS”“有限差分法”“ARIMA 模型”“SIR 模型”“遗传算法”。
- **方法类关键词（1-2 个）：**求解与分析手段，如“熵权法”“灵敏度分析”“蒙特卡洛模拟”“交叉验证”。
- **领域类关键词（1 个）：**应用领域，如“热防护服装”“供应商评价”“交通信号配时”“传染病预测”。

- 总数：5-6 个关键词，用分号分隔。
- **禁忌：**不要使用”数学建模””模型””分析””优化””问题””研究””方法”等泛化词汇作为关键词——这些词没有任何检索价值。

10.7 常见扣分项与急救措施

表 18: 摘要常见扣分项与急救措施

扣分项	表现	急救措施
无具体数字	”效果良好””取得不错结果”	每个问题后面强制写一个带数值的结果，还没算出来先留 {} 标记
字数不足	<600 字，显得内容单薄	每个问题至少 180 字，方法 + 过程 + 结果三段完整展开
套话充数	”随着科技发展””具有重要理论意义”	删除所有套话，替换为具体做了什么 + 怎么做的 + 结果多少
方法遗漏	正文用了三个模型，摘要只提了两个	正文和摘要的方法列表一一对应，提交前逐项核对
结构残缺	缺总起段或总结段	五段式（总起 + 三问 + 总结）是国赛标配，缺一段即扣分
摘要-正文不一致	摘要里的数字和正文对不上	摘要必须是正文写完之后写的，写完后回正文验证每个数字
关键词太泛	”数学建模””分析””优化”	必须写具体算法全称和数据领域

10.8 摘要提交流程与自检清单

摘要必须在正文全部完成后撰写（而非赛前写好框架填空）。原因：摘要中的每个数字、每个方法名都必须与正文最终版本完全一致。

提交前自检（10 条）：

✓ 总结

1. 不看正文，仅读摘要——能否完整说出”用了什么方法、怎么做的、结果是什么”？
2. 每个问题段落是否跟了至少一个具体数值？
3. 是否出现”首先、接着、然后、最后”四个逻辑连接词？

4. 五段结构（总起 + 三/四问 + 总结）是否齐全？
5. 字数在 800-1200 之间？
6. 是否出现“我们”“笔者”“可能”“大概”“随着科技发展”等禁忌词？
7. 正文中出现的每个模型名称，摘要中是否都有提及？
8. 摘要中的每个数字，在正文中是否都能找到出处且完全一致？
9. 关键词是否覆盖“模型 + 方法 + 领域”三要素？是否为 5-6 个？
10. 是否包含灵敏度分析/误差分析的结果？（如有此项则摘要中必须提及）

11 团队分工与时间管理

核心：国赛输在时间管理上的队伍远比输在数学水平上的多。**三天节奏踩对 + 三人并行不互等 = 成功一半。**

11.1 三天标准时间线

表 19: 国赛 74 小时作战计划

时段	核心任务	底线
第一天		
18:00–20:00	读题、选题讨论	—
20:00–22:00	定题 + 查文献	22:00 前定题
22:00–01:00	第一问建模思路 + 论文框架	第一问思路必须出来
第二天		
08:00–12:00	第一问建模 + 求解	中午前第一问出结果

表 19: 国赛 74 小时作战计划 (续)

时段	核心任务	底线
13:00– 18:00	第二问建模 + 求解	不等模型——编程先用假数据写代码
19:00– 01:00	写作 + 完善前两问	前两问必须完成
第三天		
08:00– 12:00	第三问建模 + 求解	最难问放上午
13:00– 18:00	灵敏度分析 + 模型检验 + 全部收尾	必做灵敏度
19:00– 01:00	完善结果、画图、论文初稿	所有求解完毕
提交日 (第三天之后)		
08:00– 12:00	写摘要 + 整合论文	早上一起来就写摘要
13:00– 16:00	反复改论文 + 排版	通读 ≥ 3 遍
16:00– 19:00	格式检查 + 查重 + 导出 PDF	留 1h 缓冲
19:00 前	提交	绝不拖到最后 1 小时

11.2 三人角色分工

表 20: 三人分工方案

建模手	编程手	写手
查文献、定方法 推导公式、写模型 描述	Python/MATLAB 实现 跑数据、调参、画图	论文框架 + 引言 写各节文字内容

表 20: 三人分工方案（续）

建模手	编程手	写手
设计灵敏度方案	输出结果表格	排版 + 摘要
核心原则：每人都是”T 型”——有主责，但也懂别人的活。 建模手能写代码片段，编程手能改公式，写手能看懂模型。		

11.3 最致命的三个配合问题

1. **互相等。**”我在等建模手给公式”→”我在等编程手出结果”→”我在等写手整合”。这是死循环。**破法：**建模手给大致方案 → 编程手立刻用假数据写代码框架 → 写手立刻写论文框架。不等。
2. **建模手换了模型不通知。**第二天下午建模手突然说”昨天那个模型不对，换了一个”。编程手和写手的工作全白费。**破法：**换模型必须三人碰头讨论，评估对代码和论文的影响，一起决定。
3. **代码跑崩了没人能修。**编程手写到凌晨 3 点代码跑崩，其他人看不懂、帮不上忙。**破法：**建模手必须能看懂编程手的关键代码段；遇到难题先用简化模型出结果，论文能交卷再回来优化。

11.4 协作工具

- 论文协作：Overleaf (LaTeX, 实时协同) / 腾讯文档 (Word, 实时协同)
- 即时沟通：微信群开免打扰，只在每 2 小时碰头时讨论。平时各自干活。
- 每 2 小时碰头 (5 分钟)：各自说——做了什么、遇到什么问题、接下来 2 小时做什么。

11.5 紧急预案

✓ 总结

- LaTeX 编译失败 → 切 Word 模板（提前备好，格式已设好）
- 代码跑不出 → 先出简化版本结果，论文写明”采用简化模型”，后面再优化
- 有人心态崩了 → 另两人立刻接手他的活，”先交卷再哭”
- 电脑/网络坏了 → 代码 + 论文全程云端备份

- 题目太难做不动 → 查文献找简化版，先出结果再优化
- 提交前网站崩了 → 提前 1h 提交，崩了就截图联系组委会应急通道

AI 是加速器，不是替代品。AI 负责提速（分析赛题、写代码框架、画图），你负责把关（判断对不对、解释清不清、结果对不对）。用好了省一半时间，用错了白忙一场。**AI 生成的任何内容你都要逐行看过、能解释清楚——答辩时如果答不上来，直接露馅。**

12.1 AI 在国赛中的定位

表 21: AI 能做什么 vs 不能做什么

AI 擅长的（让它做，你审）	AI 不擅长的（必须你做）
快速提取题目关键信息和数据要求	判断模型是否合理、假设是否恰当
生成 Python/LaTeX 代码框架	理解题目的深层含义和隐含约束
编写 matplotlib 绘图代码	解释”为什么选这个模型”
批量数据清洗和转换代码	创新性的建模思路和方法组合
检查论文语法错误和格式	保证结果的正确性（AI 经常算错）
中英文翻译和文字润色	写摘要（AI 写的摘要一眼假，直接扣分）

12.2 赛题分析：怎么用 AI 分析题目

拿到题目的第一个小时最关键。以下是经过验证的分析流程：

第 1 步：快速通读（10 分钟，不用 AI）

三人各自通读所有题目，凭自己的理解标注”能做””做不了””需要查资料”。这一步不要 AI——你需要先形成自己的初步判断，否则容易被 AI 带偏方向。

第 2 步：AI 辅助深度分析（每道候选题目 10-15 分钟）

将题目全文粘贴给 AI，使用以下 prompt：



例子与推导

赛题分析 Prompt:

你是数学建模竞赛教练。请分析以下国赛题目并给出建议。

【粘贴题目全文】

请完成：

1. 用1句话概括每个小问的核心任务。
2. 每个小问列出2-3种可行方法，从简单到复杂排列，标注每种方法的适用条件和局限性。
3. 指出本题的3个关键难点，以及每个难点的应对思路。
4. 推荐一个整体建模框架（问题之间是什么关系？是否前一问的结果是后一问的输入？）。
5. 列出需要的数据类型和数据量级。如果附件没提供某个数据，给出替代方案。
6. 这道题对队伍有什么要求？（需要哪方面的知识储备？编程难度？）

要求：方法推荐以简单实用优先。复杂方法只在简单方法不够用时再考虑。

第3步：交叉验证

将同一道题给两个不同的 AI 工具分析，对比结论。如果结论一致——可靠。如果有分歧——你们需要自己判断哪个更合理，AI 只能提供参考，**最终决定必须是你们自己做的。**

第4步：定题后的深入分析

定题后，用以下 prompt 细化建模方案：



例子与推导

定题后细化 Prompt:

我们选定[题号]，初步方案是：

- 问题一用[方法A]
- 问题二用[方法B]
- 问题三用[方法C]

请针对这个方案：

1. 方法A的完整数学推导步骤是什么？给出核心公式和变量定义。
2. 方法A的Python实现需要哪些库？给出完整的代码框架（含函数定义、输入输出说明）。
3. 方法A可能遇到的3个问题及解决方案。
4. 问题一的结果如何作为问题二的输入？数据流转逻辑梳理清楚。
5. 整个方案的风险点在哪里？哪个环节最可能出问题？

12.3 代码生成：怎么用 AI 写代码不出错

原则：不要让 AI 从零写一个你不知道怎么写的程序。正确的做法是——你知道程序的逻辑步骤，让 AI 填空。

代码生成的标准流程：

1. 你先设计函数接口：函数名、参数、返回值、处理步骤——你先写清楚
2. AI 生成实现代码：把接口描述给 AI，让它填充实现
3. 你逐行审查：每行代码都要看懂，拿假数据先跑一遍
4. 对比标准结果：如果能手动算一个简单情况，和 AI 代码的输出对比



例子与推导

代码生成 Prompt 模板：

请用Python实现以下函数。

函数名： `entropy_topsis(data, benefit_cols)`

输入：

- `data`: $n \times m$ 的 `numpy ndarray`, n =方案数, m =指标数
- `benefit_cols`: `list`, 效益型指标（越大越好）的列索引

处理步骤：

1. 正向化：非效益型指标取其倒数，统一为"越大越好"
2. 标准化：`min-max`标准化到 `[0,1]`
3. 熵权法：计算每列的信息熵，信息熵越小权重越大
4. TOPSIS：构造加权标准化矩阵，计算每个方案到最优/最劣解的距离

5. 返回综合得分（相对贴近度）

输出：

- `scores`: 长度为`n`的1D array，每个方案的综合得分
- `weights`: 长度为`m`的1D array，每个指标的权重

要求：

- 仅依赖 `numpy`
- 每行代码加中文注释
- 给出一个简单示例验证正确性
- 考虑数值稳定性（除零保护）

AI 代码的常见陷阱——必须检查的三点：

1. **AI 会编造不存在的参数。**比如 `sklearn` 的某函数根本没有某个参数，AI 却用了。检查方法：每个不熟悉的参数去官方文档确认。
2. **AI 会写逻辑正确但效率极差的代码。**比如在循环里反复创建大数组。长数据要测试性能。
3. **AI 会”不小心”修改关键逻辑。**你让它优化某处代码，它可能把其他地方也改了。只复制你需要的改动，不要整段替换。

12.4 绘图：怎么用 AI 画图

绘图是 AI 最擅长的领域之一。标准流程：

Listing 10: 绘图 Prompt 模板

请用 `matplotlib` 画以下图：

- 类型：[散点图/折线图/柱状图/热力图/雷达图/箱线图]
- 数据：[描述数据格式和范围]
- X轴：[标签，范围]
- Y轴：[标签，范围]
- 图表要求：中文字体(`SimHei`)，分辨率 `150dpi`，保存为 `PNG`
- 配色方案：[`tab10`/`viridis`/自定义颜色]
- 特殊要求：[标注数据点/添加趋势线/添加图例位置/添加网格]

AI 绘图的最佳实践：

- 先用 AI 生成初版代码 → 运行看效果 → 告诉 AI 哪里需要改 → 迭代 2-3 轮得到最终图
- plt.savefig 时指定 dpi=150，文件名用中文（见名知义）
- AI 生成的图你可能还要手动调字体大小、标注位置，这是正常的

12.5 论文润色：怎么用 AI 改论文

AI 可以帮你检查语法、统一术语、提高表达精度。但**绝对不能让 AI 写整段论文**——AI 写的文字有明显的模式化痕迹，查重系统现在能检测 AI 生成文本。

正确的方式：

1. 你先写完论文段落——用自己的话写，哪怕粗糙一点
2. 把段落给 AI：“请检查这段文字中的语法错误、术语不一致、表达不清的地方，逐一标注建议修改。但不要改我的意思，不要添加我没有的信息。”
3. 你逐条审查 AI 的建议——同意的改，不同意的保留原样

12.6 不可逾越的红线

✓ 总结

- **不用 AI 写摘要**——AI 摘要模板味重、空洞，评委会直接识别并扣分
- **不直接复制 AI 代码**——逐行看懂、本地跑通、用假数据验证
- **不用 AI 查文献**——AI 会编造不存在的论文和作者名。文献只能去知网/Google Scholar
- **AI 生成内容占论文比例不超 30%**——核心建模思路 and 关键推导必须自己完成
- **所有 AI 生成的结果（代码输出、图表数据）必须验证**——AI 可以算错、可以用错公式

12.7 推荐 AI 工具组合

表 22: 国赛 AI 工具组合

工具	擅长	使用场景
Claude / GPT-4	综合推理、赛题分析	拿到题目先分析，定方案前交叉验证
Claude Code / Cursor	写代码、调试	实现模型、调 bug、优化效率
DeepSeek	数学推理	公式推导、算法逻辑验证
GitHub Copilot	代码补全	写重复性代码时加速

13

国赛易违规条款分析

一句话核心：国赛每年有 5%-10% 的队伍因违规被取消资格——不是模型不行，是规矩没搞清。**查重、引用、格式、提交**——每条都可能是雷。

13.1 查重红线（碰了就淘汰）

表 23: 查重红线一览

查重类型	红线标准	检测方式
论文正文查重	全文重复率 > 25%	系统自动比对（知网/万方）
代码查重	代码相似度 > 50%	MOSS 等代码查重系统
附录查重	与知网/已发表论文重复	和正文一起查
跨队查重	两队论文高度相似	全省论文交叉比对

特别注意：

- 很多学校图书馆买了查重系统，赛前可以自查一次
- 公式推导部分最容易重复（大家都抄同一本教材）——用自己的话改写
- 引言/背景部分最容易不小心抄——必须用自己的话写

13.2 引用规范

引用原则

凡是别人的观点、数据、公式、图表，**必须标注出处**。不标 = 抄袭。

哪些必须引用？

- 引用的数据（如“据国家统计局 2023 年数据……”）
- 引用的公式（如“SIR 模型最早由 Kermack 和 McKendrick 提出 [1]”）
- 参考的建模思路（如“本文借鉴了文献 [2] 中的层次分析法框架”）
- 引用的图片（在标题中注明“图片来源：文献 [3]”）

哪些不需要引用？

- 数学常识（如线性回归的公式、导数的定义）
- 你自己推导的公式
- 你自己画的数据图
- 你自己写的代码

引用格式（GB/T 7714）：

- 期刊：[1] 作者. 题名 [J]. 刊名, 年份, 卷 (期): 页码.
- 书籍：[2] 作者. 书名 [M]. 出版地: 出版社, 年份.
- 网络资源：[3] 作者. 题名 [EB/OL]. URL, 发布日期.

13.3 代码查重（最容易忽略的雷区）

总结

代码查重 ≠ 论文查重。即使论文是自己写的，代码如果是复制开源项目的，也可能被查出。国赛现在使用 MOSS 等代码查重系统。

怎样算代码违规？

- 直接复制 GitHub 上的完整项目代码
- 与另一队代码高度相似（变名调用不同但逻辑完全一致也会被查）
- 用 AI 生成代码后不加修改直接使用

怎样不算违规？

- 调用公开库（sklearn、numpy）——所有人都用
- 参考了网上的思路，但自己重写实现
- 公共代码模板（如数据读取、画图模板）

安全做法：

1. 代码自己写/自己改——不要直接复制粘贴
2. 变量名用自己的命名习惯
3. 代码结构有自己的逻辑（不要和网上教程一个模子）
4. 附录中可以附核心代码，但不是必须附全部

13.4 格式违规（不起眼但扣分）

表 24: 常见格式违规项

违规项	正确做法
论文中写学校/队员名	论文中绝对不能出现学校名和人名
超页数	正文不超过 20 页（含附录看具体要求）
页边距故意缩小	按规范 2.5cm，不要耍小聪明
字体不统一	正文/标题/公式字体统一
图表无编号	每个表/图必须有编号 + 标题
PDF 不清晰	用矢量图或高分辨率 PNG

13.5 提交环节（最后一公里翻车）

✓ 总结

- **提前 1 小时提交：**最后半小时网站必崩。每年都有大量队伍因超时未提交而 0 分。
- **确认文件格式：**PDF（不是 Word 不是图片不是压缩包），文件大小不超标。
- **确认文件完整：**下载自己提交的文件，打开确认能正常显示。
- **MD5 校验：**提交前记录文件 MD5，提交后核对——确保上传过程无损。
- **提交失败怎么办：**立刻截图 + 联系组委会（比赛通知里通常会留应急联系方式）。

- **不要等到最后**：周日 17:00 就可以提交了，别等到 19:50。

13.6 违规后果速查

表 25: 违规类型与后果

违规行为	后果
查重率 > 25%	取消评奖资格
代码查重 > 50%	取消评奖资格
论文出现学校/姓名	取消评奖资格（部分省份）
未按时提交	0 分
提交格式错误（非 PDF）	0 分
抄袭他人论文	取消资格 + 通报学校
代替他人参赛	取消资格 + 处分

使用说明

- 本速成课共 **13 章**：6 类模型（第 2-7 章）+ 7 项技能（第 1、8-13 章）。
- 模型章聚焦 **通用解题套路**（适用判断 → 模型速查 → 解题流程 → 代码模板 → 常见坑）。
- 技能章聚焦 **标准模板/清单**（为什么重要 → 模板 → 常见错误 → 速查清单）。
- 新增章节：复制 chapters/_ 章节模版/ → 改名 → 填内容 → 加 \input。
- 修改章节顺序只需调整 \input 的先后。
- 每章自带 images/ 和 生图代码/ 文件夹。